Cognitive Computation (2026) 18:103
https://doi.org/10.1007/s12559-026-10619-1

REVIEW

From Language Models to Agentic AI: A Survey of Autonomous, Action-Enabled, and Collaborative LLM Agents

Sparsh Bajoria[1] · Shreyanshu Ranjan[1] · Adhitya M[1] · Vineet Suri[2] · Vikas Hassija[1] · Vinay Chamola[2] · Amir Hussain[3]

Received: 8 March 2026 / Accepted: 20 June 2026
© The Author(s) 2026

Abstract

Large Language Model (LLM)–based agents are rapidly evolving from passive assistants into autonomous, tool-using, and collaborative systems capable of executing complex, long-horizon tasks across web, software, and physical environments. However, the current literature remains fragmented, with inconsistent terminology, ad hoc architectures, and limited evaluation standards, making it difficult to compare systems or deploy them reliably in real-world settings. This paper presents a unified, taxonomy-driven, and deployment-oriented survey of agentic AI systems, synthesizing recent advances through a modular reference architecture and a four-dimensional taxonomy that characterizes agents along the axes of autonomy, tool use, collaboration, and safety–governance. We systematically analyze representative single-agent, tool-augmented, and multi-agent frameworks within this taxonomy, highlighting design trade-offs, capability scaling patterns, and recurring failure modes. Beyond architectural analysis, we review emerging evaluation methodologies that move beyond static benchmarks to assess agent behavior, robustness, grounding, and operational cost in interactive environments. Importantly, the survey emphasizes practical considerations for enterprise and safety-critical deployment, including access control, human-in-the-loop oversight, and policy enforcement. By unifying conceptual foundations with empirical trends and deployment constraints, this work provides a structured roadmap for researchers and practitioners to design, evaluate, and govern next-generation LLM-based agentic systems.

Keywords Large language models · AI agents · agentic AI · Multi-agent systems · autonomous systems

Introduction

Large language models (LLMs) have become the dominant foundation for natural language processing, code generation, and many general-purpose reasoning tasks [1–6]. Initially, LLMs were deployed primarily as *passive* components: a user provided a prompt, and the model returned a single text completion [2, 5]. In the last two years, this interaction pattern has shifted toward *agents* in which an LLM (or a set of LLMs) continuously *observes*, *reasons*, *acts*, and *adapts* in dynamic environments [7–11]. This transition is enabled by advances in prompting and structured reasoning that treat

✉ Vikas Hassija
vikas.hassijafcs@kiit.ac.in

Sparsh Bajoria
2329214@kiit.ac.in

Shreyanshu Ranjan
2305013@kiit.ac.in

Adhitya M
23051968@kiit.ac.in

Vineet Suri
2023ac05031@wilp.bits-pilani.ac.in

Vinay Chamola
vinay.chamola@pilani.bits-pilani.ac.in

Amir Hussain
amir.hussain@kfupm.edu.sa

[1] School of Computer Engineering, KIIT, Bhubaneswar 751024, India

[2] Department of Electrical and Electronics Engineering, Birla Institute of Technology and Science, Pilani, Pilani Campus, Vidya Vihar, Pilani, Rajasthan 333031, India

[3] SDAIA-KFUPM Joint Research Centre for AI, Information and Computer Science Department, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia

Published online: 24 August 2026

LLMs as general-purpose decision policies rather than mere text generators [12–15].

This agentic shift is visible in three converging lines of work. First, *LLM-based autonomous agents* embed the model in a decision loop that iteratively plans, executes actions, and incorporates feedback, often over long horizons [7, 8, 16–18]. Second, *tool-using agents* extend the model with explicit interfaces to external tools—such as search engines, databases, code execution sandboxes, and enterprise APIs—allowing the agent to overcome limitations in factual recall, calculus, and environment interaction [19–22]. Third, *LLM-based multi-agent systems* (LLM-MAS) organize multiple LLM agents that coordinate, compete, or debate to solve complex tasks and simulate social or organizational dynamics [23, 24].

Recent advancements in agentic AI are enabling the integration of intelligent decision-making with real-world systems through digital twins, thereby facilitating autonomous and adaptive system behavior[25].

These developments have triggered a surge of survey papers. Wang et al. review LLM-based autonomous agents and propose a unified framework focusing on construction, applications, and evaluation [7]. Xi et al. trace the conceptual evolution of agents and discuss single-agent, multi-agent, and human–agent cooperation scenarios [8]. Guo et al. and Li et al. provide systematic overviews of LLM-based multi-agent systems, with emphasis on agent profiling, communication, and application domains [23]. More recent works survey memory mechanisms, feedback designs, and specific application verticals [16–18]. While these surveys are highly valuable, they typically focus on one axis at a time—autonomy, tool use, or multi-agent collaboration—and rarely connect these aspects into a single engineering-oriented picture.

In practice, however, modern LLM agent systems seldom fit into a single axis. A production-ready agent may need to autonomously plan tasks, call multiple internal APIs, coordinate with other agents, and operate under strict governance and audit requirements [19, 20, 23, 26]. Without a unified view across autonomy, tool use, collaboration, and deployment considerations, it becomes difficult for researchers and practitioners to (i) understand the design space, (ii) position new systems within that space, and (iii) identify gaps in current evaluation methods and benchmarks [27–30].

Throughout this paper, we use the term "LLM-based agents" to refer to specific system implementations, while "agentic AI" denotes the broader paradigm encompassing autonomy, tool use, and collaboration.

Motivation

The rapid growth of the agentic AI literature creates both opportunities and confusion. On the positive side, new architectures, frameworks, and benchmarks appear almost weekly, accelerating innovation in reasoning, planning, tool use, and collaboration [12, 14, 15]. On the negative side, terminology is inconsistent (e.g., *agents*, *tools*, *orchestrators*, *workers*, *co-pilots*), and there is substantial overlap among systems with different names but similar structural patterns [7, 8, 23, 26]. Existing surveys provide important slices of this landscape, but they are not yet sufficient as a single "entry point" for engineers who need to build or evaluate LLM agents in practice [16, 28].

More concretely, several gaps motivate this survey:

- **Fragmented views:** Current surveys typically specialize in either single-agent autonomy [7], the general concept of LLM-based agents [8], or LLM-based multi-agent systems [23]. There is a lack of a *joint* perspective that treats autonomy, tool use, and collaboration as first-class, interacting dimensions.
- **Missing engineering-level taxonomy:** Most existing taxonomies are conceptual or domain-specific [7, 16, 23]. Practitioners need an engineering-oriented taxonomy that can be used to characterize concrete systems in terms of autonomy level, tool-integration pattern, collaboration structure, and safety/oversight mechanisms [28, 29].
- **Evaluation blind spots:** Many works rely on task-level success metrics alone, using general-purpose benchmarks or small custom suites [27, 30]. Process-level behavior (e.g., tool-call traces, collaboration dynamics), safety and robustness, and operational cost are often under-specified or evaluated in ad hoc ways [29].
- **Disconnect between research and deployment:** Industrial systems increasingly integrate LLM agents into governed workflows, but deployment and governance patterns are only partially reflected in existing academic surveys [28, 30].

These gaps suggest the need for a survey that is both *conceptually unifying* and *deployment-aware*, synthesizing the research literature on autonomous agents, tool-using agents, and multi-agent systems with emerging practices in evaluation, safety, and governance [7, 23, 27, 29].

Scope and Research Questions

To address the above gaps, this survey focuses on LLM-based agents that satisfy at least one of the following:

1) **Autonomous decision-making:** the system embeds one or more LLMs in a control loop that iteratively plans, executes, and updates based on feedback, beyond single-turn question answering [7–10].

2) **Explicit tool use:** the agent has an explicit interface to external tools or APIs (e.g., search, calculators, databases, code execution, enterprise systems), and tool calls are integral to its behavior [19–22].
3) **Multi-agent collaboration:** the system involves multiple LLM-based agents that interact via natural or structured language to solve tasks collectively or simulate social dynamics [23, 24].

Within this scope, we organize the survey around the following research questions (RQs):

- **RQ1:** How can we describe LLM agents in a unified modular framework that covers perception, memory, reasoning/planning, tool interfaces, actuation, and feedback/oversight [12, 14, 16–18]?
- **RQ2:** Along which dimensions can we systematically categorize LLM agents in terms of autonomy, tool use, collaboration structure, and safety/governance [7, 23, 28, 29]?
- **RQ3:** What are the dominant design patterns and trade-offs for autonomous LLM agents, tool-using agents, and LLM-based multi-agent systems [9, 10, 19, 21, 24]?
- **RQ4:** How are LLM agents evaluated today, and which aspects of their behavior remain under-evaluated [27, 29, 30]?
- **RQ5:** What open challenges and research opportunities remain for building trustworthy, scalable LLM agents deployed in real-world environments [7, 8, 23, 28]?

Our Contributions

This survey makes the following contributions:

1) **Unified agent framework:** We propose a modular reference architecture for LLM agents that separates perception, memory, reasoning/planning, tool interface, actuation, and feedback/oversight components, drawing on recent patterns in reasoning, memory, and self-improvement [12, 14–18].
2) **Four-dimensional taxonomy:** We introduce an engineering-oriented taxonomy along four interacting dimensions: autonomy level, tool awareness and integration, collaboration structure, and safety/governance mechanisms, synthesizing insights from both single-agent and multi-agent surveys [7, 8, 23, 28, 29].
3) **Integrated view across three axes:** We jointly review autonomous LLM agents, tool-using/tool-making agents, and LLM-based multi-agent systems, connecting ideas that are often studied in isolation and aligning them with the unified framework [19, 21, 23, 24].
4) **Evaluation and benchmarks:** We synthesize existing benchmarks and evaluation methodologies for agents, emphasizing the distinction between task-level and process-level metrics, and highlighting gaps in robustness, safety, and cost evaluation [27, 29, 30].
5) **Deployment-aware perspective:** We complement academic literature on foundation models and agents with broader discussions of opportunities, risks, and governance for large-scale deployments [28–30].

In addition, we clarify how the proposed taxonomy can be applied in practice by system designers across different application domains, highlighting its utility for comparative analysis and system-level decision-making.

Paper Organization and Positioning

Before outlining the structure of this survey, we first position our contribution with respect to existing reviews on LLM-based agents and multi-agent systems. Table 1 provides a comparative summary of representative surveys, highlighting the dimensions they cover across autonomy, tool use, and multi-agent collaboration, and indicating whether they adopt an explicit engineering-oriented taxonomy or address real-world deployment considerations [7, 8, 23, 26, 28]. This comparison motivates the need for a unified and deployment-aware perspective, as most existing surveys focus on isolated aspects of agent behavior (e.g., reasoning or collaboration) without jointly analyzing architectural patterns, decision-making mechanisms, and governance constraints [16–18, 29].

The remainder of this paper is organized as follows. Section "Background and Terminology" introduces the foundational background, reviewing advances in large language models and generative AI, extending to multimodal and vision–language models, and establishing precise definitions of agentic AI used throughout the survey [1–8]. Section 3 presents a unified modular framework for LLM agents and introduces a four-dimensional taxonomy based on autonomy, tool integration, collaboration, and safety/governance.

Sections "Autonomous LLM Agents", "Tool-Using and Tool-Making LLM Agents", and "LLM-Based Multi-Agent Systems and Collaboration" examine key system paradigms in detail. Section "Autonomous LLM Agents" focuses on autonomous LLM agents, including planning, self-reflection, and memory-augmented behavior, building on reasoning and feedback mechanisms such as chain-of-thought, tree/graph-based reasoning, and self-improvement loops [12, 14, 15, 17, 18]. Section "Tool-Using and Tool-Making LLM Agents" discusses tool-using and tool-making agents, highlighting tool integration patterns, reliability considerations, and system design trade-offs [10, 19, 21]. Section "LLM-Based Multi-Agent Systems and Collaboration" explores LLM-based multi-agent systems, including coordination mechanisms,

Table 1 Conceptual comparison of plain LLMs, LLM-based assistants, LLM agents, and LLM-based multi-agent systems (LLM-MAS).

System type	Substrate	Control loop	Tool access	No. of agents	Typical tasks
Plain LLM model [31]	Single input text (no persistent state)	Single forward pass; no explicit perception–action cycle	None or implicit (parametric knowledge only)	0 (no explicit agent)	Text completion, classification, standard LM benchmarks
LLM-based assistant [32]	Short-turn chat or question–answer exchanges	Single- or few-turn dialog loop driven by user prompts	Optional retrieval / search; occasional function calls or plug-ins	1 assistant interacting with a human user	QA, summarization, drafting, interactive helpdesks
LLM agent [32]	Episodic or persistent task environment with memory	Explicit perception–reasoning–action loop with planning and feedback	Explicit tool APIs (search, code, databases, OS, simulators)	1 autonomous agent (possibly with human-in-the-loop)	Web navigation, coding agents, workflow automation, open-ended exploration
LLM-MAS (multi-agent) [32]	Shared environment or communication substrate (blackboard, message bus, simulator)	Coordinated loops across multiple agents with protocols and roles	Explicit tools *and* communication APIs between agents and services	>1 agents with specialized roles (planner, worker, critic, user proxy, etc.)	Collaboration, debate, social simulation, complex enterprise workflows

communication protocols, and emergent behaviors across collaborative agent settings [23, 24, 26].

Section "Cross-Cutting Failure Modes and Limitations" analyzes cross-cutting failure modes that arise across agentic systems, including error accumulation, misalignment, robustness issues, and coordination breakdowns. Section "Evaluation and Benchmarks for LLM Agents" reviews existing evaluation methodologies and benchmarks, emphasizing the need for process-level, safety-aware, and cost-aware evaluation metrics [27, 30]. Section "Open Challenges and Research Agenda" outlines open research challenges and future directions, including scalable architectures, trustworthy deployment, and human–agent collaboration paradigms [7, 8, 23].

In application domains such as intelligent transportation systems, large language models are increasingly explored to enhance decision-making, traffic management, and system-level optimization [33]. Multimodal data fusion plays a key role in modeling complex real-world systems [34]. Capturing non-periodic patterns is essential for accurate prediction in dynamic environments [35]. Cross-modal retrieval techniques enable alignment between heterogeneous data sources [36].

Section "11" discusses the limitations of current approaches and the scope of this survey, including challenges related to data quality, scalability, and deployment constraints. Finally, Section XI concludes the paper and summarizes key insights and implications for future agentic AI research and deployment, while revisiting fundamental challenges such as hallucinations, bias, and reliability in high-stakes applications [28, 29, 37].

Positioning Relative to Existing Surveys

Recent surveys have provided valuable overviews of autonomous LLM agents, tool-augmented reasoning, and multi-agent collaboration [7, 8, 23]. Wang et al. [7] focus on architectural patterns and applications of autonomous agents, while Xi et al. [8] provide a broad conceptual overview of the emergence of agentic behaviors in LLMs. Other works emphasize evaluation benchmarks or multi-agent coordination in isolation [24, 27, 38, 39].

In contrast, this survey contributes a unified design-space perspective that jointly models autonomy, tool interaction, collaboration structure, and governance/safety [40, 41]. It explicitly connects architectural choices to evaluation criteria and deployment risks [28–30, 42].

Recent surveys have provided valuable perspectives on agentic AI, including architectural frameworks, cognitive capabilities, and application domains. These works often cover overlapping dimensions such as autonomy, memory, and system design. However, they typically emphasize conceptual or domain-specific viewpoints.

In contrast, this survey focuses on an engineering-oriented integration of autonomy, tool interaction, collaboration structure, and safety/governance into a unified analytical framework. Our goal is not to introduce entirely new dimensions, but to systematically combine and operationalize them to support comparative analysis and deployment-oriented system designs [43–45].

Survey Methodology

To ensure coverage and reproducibility, this survey follows a structured literature review process. We collected relevant works published between 2022 and early 2026 from IEEE Xplore, ACM Digital Library, arXiv, and Semantic Scholar using keyword combinations such as "LLM agents", "agentic AI", "tool-using language models", and "LLM-based multi-agent systems". We included peer-reviewed papers, widely cited preprints, and authoritative industrial reports

that introduced novel architectures, evaluation frameworks, or deployment practices for LLM-based agents. Works focusing exclusively on static language models or non-LLM symbolic agents were excluded. Each selected system was analyzed and annotated along the four taxonomy dimensions introduced in Section III—autonomy (A), tool use (T), collaboration (C), and safety/governance (S)—based on descriptions in the original publications and accompanying documentation. This methodology supports consistent comparison across heterogeneous systems and highlights under-explored regions of the design space.

Background and Terminology

A recent survey by Bandi et al [46] provides a broad overview of agentic AI definitions and frameworks; however, it does not explicitly unify autonomy, tool integration, collaboration, and governance into a single engineering-oriented taxonomy as proposed in this work.

We begin with a brief view of large language models (LLMs) as general-purpose policies, then recall classical notions of intelligent agents and multi-agent systems. Finally, we bridge these perspectives to define *agentic AI* and harmonize terminology that is used inconsistently across the literature [47].

Large Language Models as General-Purpose Policies

Modern LLMs, typically based on transformer architectures, are trained on large corpora to predict the next token given a context window [48]. Despite this simple training objective, LLMs have demonstrated strong capabilities in instruction following, in-context learning, and multi-step reasoning.

A major challenge in scaling intelligent systems remains learning under low-resource conditions[49]. This has led many works to view an LLM as a *general-purpose policy* that maps *prompts* (states and instructions) to *actions* (text outputs, tool calls, or structured messages) [50].

In their simplest form, LLM applications follow a one-shot pattern: a user provides an instruction x, the model produces a response $y = f_\theta(x)$, and the interaction terminates [51]. Agentic settings instead embed the LLM inside a recurrent loop: the model receives a sequence of observations and interaction history, selects an action (which may be a tool call or environment action), observes the result, and repeats [52]. This view connects LLM agents to classical sequential decision-making and reinforcement learning, where a policy $\pi(a \mid s)$ maps states to actions over time [53].

Recent research has increasingly integrated transformer architectures with reinforcement learning for decision-making tasks[54].

Classical Intelligent Agents and Multi-Agent Systems

Long before LLMs, classical AI defined an *intelligent agent* as an entity that perceives its environment through sensors and acts upon that environment through actuators to achieve goals [55]. An agent is rational if it selects actions that maximize expected performance according to a utility function, given its knowledge and perceptual history [56]. The canonical agent loop is: observe → update internal state → select action → act → observe again [57].

This transition from classical perception–action loops to LLM-based iterative reasoning and action cycles is illustrated in Fig. 1.

Multi-agent systems (MAS) extend this idea to multiple interacting agents that may cooperate, compete, or both [58]. Agents in MAS can differ in their capabilities, knowledge, and goals [59]. Coordination mechanisms include explicit communication protocols, market-based mechanisms, centralized planners, and emergent behavior in decentralized systems [60] (Fig. 2).

Formal approaches to multi-agent reinforcement learning have been proposed using graph-based Q-learning frameworks [61].

These frameworks have been widely studied in robotics, distributed control, and game-theoretic settings [62].

Classical MAS research provides concepts that are directly relevant to LLM-based multi-agent systems: notions of roles and organizations, protocols for negotiation and coalition formation, and analysis of emergent phenomena such as cooperation, free-riding, and collusion [63]. In this survey, we reuse these concepts but with LLMs serving as the cognitive core of agents [64].

From Classical Agents to Agentic AI

Recent work under the label *agentic AI* revisits classical agent concepts in the context of foundation models [65]. While definitions vary across papers and reports, most agree on a few key features: (i) goal-directed behavior,(ii) autonomy over multiple steps,(iii) the ability to use tools and external resources,(iv) persistent or semi-persistent memory, and (v) adaptation based on feedback [66].

Academic surveys on agentic AI primarily synthesize research prototypes and conceptual frameworks. Industrial whitepapers and reports, in contrast, emphasize architectural patterns and governance issues for deploying agents in enterprises [67]. Both strands of work highlight that agentic systems are not limited to LLM-based agents; they also encompass symbolic agents, hybrid neuro-symbolic systems, and traditional software agents [68]. In this survey,

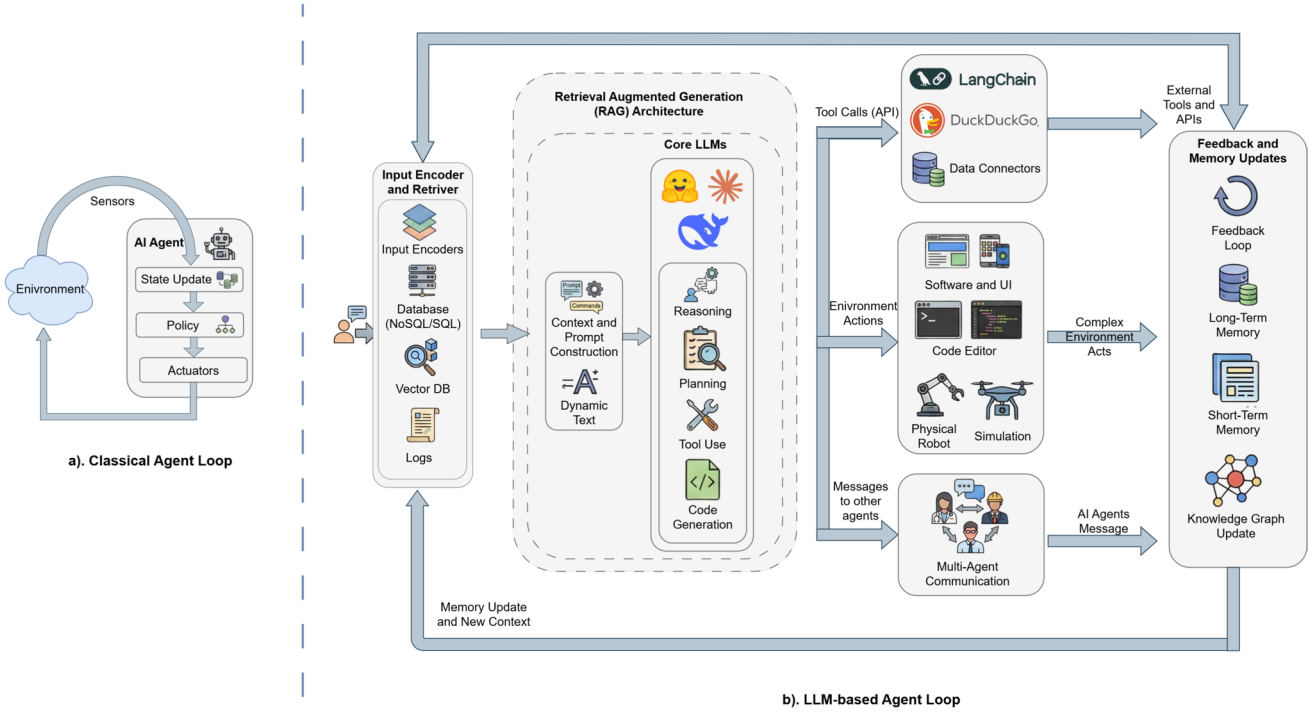


Fig. 1 Classical agent loop vs. LLM-based agent loop: (**a**) sensors → state → policy → actuators; (**b**) encoder/retriever form context, LLM generates actions (tools, environment, or messages) with feedback and memory updates

however, we focus specifically on *LLM-centric* agents, while occasionally drawing on the broader agentic AI literature for definitions and design principles [69].

Memory, Feedback, and Collaboration as Enabling Mechanisms

Three mechanisms appear repeatedly in the LLM agent literature and will recur throughout this survey [71].

Memory LLM agents often require access to information beyond the current context window [72]. Memory mechanisms can be roughly categorized into: (i) *short-term conversational memory* (keeping recent turns),(ii) *long-term episodic memory* (storing past trajectories and experiences),and (iii) *semantic memory* based on vector databases or knowledge graphs [73].Graph-based learning methods are widely used for structured knowledge representation [74].Advanced embedding techniques improve link prediction in structured knowledge graphs [75].Context-aware graph reasoning enhances relational understanding in complex systems[76]. A recent survey by Zhang et al. systematically analyzes memory designs, retrieval strategies, and evaluation criteria for LLM agents [77].

Feedback Feedback mechanisms allow agents to critique and improve their own behavior or receive signals from external sources [78]. Liu et al. categorize feedback for LLM agents into internal self-reflection, external model feedback, human feedback, and environment feedback, and discuss their integration into control loops and training [79]. Multi-agent debate and self-consistency can also be seen as particular forms of feedback, where multiple agents exchange critiques before converging on a solution [80].

Collaboration Collaboration appears in two forms:

(i) collaboration between an agent and humans (e.g., co-pilots, human-in-the-loop review), and
(ii) collaboration between multiple LLM agents in LLM-MAS frameworks. Recent work on multi-AI agent collaboration provides theoretical perspectives on coordination mechanisms and emergent behavior [81].

We treat memory, feedback, and collaboration as orthogonal mechanisms that can be combined with different autonomy levels and tool-use pattern.

Terminology Harmonization for this Survey

Because terminology is inconsistent across the literature and industry, we explicitly fix how several key terms will be used in this survey. Table 2 summarizes representative definitions of agentic AI drawn from both academic literature and selected industry reports. These sources operate at

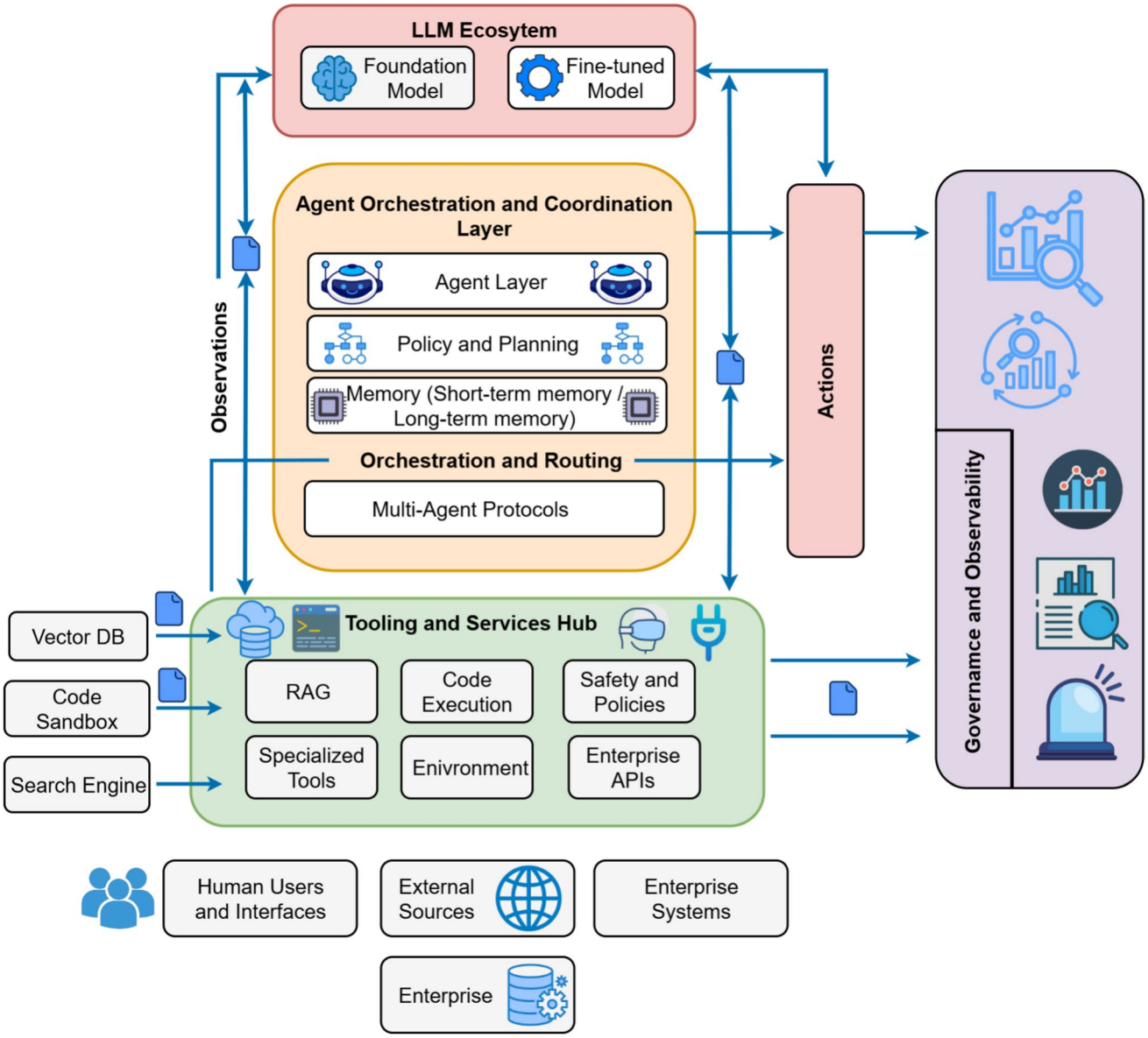


Fig. 2 Stack-oriented view of LLM agentic systems: environment at the base, tools and APIs above, an agent layer (policies, memory, orchestration, multi-agent protocols) powered by LLM cores, with monitoring, governance, and user interfaces wrapping the stack

different levels of abstraction (conceptual, architectural, and deployment-oriented), and are included to provide complementary perspectives rather than strictly comparable formulations. These definitions are intentionally pragmatic and oriented toward system design.

This harmonized vocabulary will be used consistently in the rest of the paper [46]. Whenever we use "agent" without qualification, we refer specifically to an *LLM-based agent* as defined in Table 3, not to classical symbolic agents or general agentic systems. "Agentic AI" will be reserved for high-level discussion where results and design principles apply beyond purely LLM-centric systems [65]. "Tools" will always mean explicitly invocable capabilities with structured interfaces, as opposed to implicit knowledge embedded in model parameters [47]. "Multi-agent" will refer to systems with at least two interacting LLM-based agents, regardless of whether human participants are also included [58].

Unified Framework and Taxonomy for LLM Agents

Building on the background in Section 2, we now present a unified framework that organizes LLM agents in terms of their internal modules, control loops, and external interfaces. We then develop a four-dimensional taxonomy that

Table 2 Illustrative definitions of "agentic AI" in recent literature and how this survey specializes them

Source	Definition focus	Key elements	Relation to this survey
Abou Ali et al. [40]	Conceptual and architectural survey of agentic AI across paradigms	Autonomy, memory, goal-directed behavior, adaptation, safety; includes symbolic and hybrid systems	Provides broad conceptual framing; we specialize to LLM-centric agents and connect to concrete engineering patterns.
Pati et al. [43]	Review of definitions and frameworks for agentic AI	Synthesis of definitions across research and industry; comparison with classical AI and robotics	Helps harmonize terminology; we adopt their criteria but narrow scope to LLM-based systems.
Piccialli et al. [41]	Agentic AI as autonomous goal-directed intelligence	Architectures, cognitive capabilities, application taxonomy	We reuse their high-level capabilities but organize systems along autonomy, tools, and collaboration.
Capgemini report [70]	Enterprise-oriented view of agentic AI	Agent networks, platform architectures, governance, ROI and risk	Informs our discussion of deployment and governance for LLM agents in real-world settings.

Table 3 Key terms and working definitions used in this survey

Term (this survey)	Working definition	Common synonyms in literature	Example references
LLM	A large language model used as a general-purpose conditional generator or policy, typically accessed via API for inference.	Foundation model, base model, pretrained LM.	[30, 82]
LLM-based assistant	An application where a single LLM instance interacts with a user in short dialogs, without explicit long-horizon control or structured tool orchestration.	Chatbot, copilot, assistant, conversational agent.	[30, 83]
LLM agent	A system where one or more LLMs are embedded in an explicit control loop with perception, optional memory, decision-making, and action execution (possibly via tools) to achieve goals over multiple steps.	LLM-based autonomous agent, LLM-based tool-using agent, LLM software agent.	[7–9, 19]
Tool	Any external capability (API, database, code executor, search, simulator) that the LLM agent can call through a structured interface to extend its abilities beyond text generation.	Function call, plugin, external API, tool wrapper.	[10, 19, 21]
LLM-based multi-agent system (LLM-MAS)	A system in which multiple LLM-based agents interact via messages or shared state to solve tasks jointly under explicit multi-agent protocols.	LLM multi-agent, LLM agent swarm, multi-LLM collaboration.	[23, 84, 85]
Agentic AI	An umbrella term for AI systems—including LLM agents, tool-augmented agents, and hybrids—that exhibit sustained goal-directed behavior using memory, tools, and explicit control loops.	Agentic systems, AI agents, autonomous AI.	[40, 41]

will be reused throughout the rest of this survey when analyzing concrete systems and applications.

Modular Architecture of LLM Agents

Most LLM-based agent systems can be decomposed into a small set of interacting modules, regardless of their application domain or implementation framework [7–9, 19, 23, 26]. Figure 3 depicts a generic modular architecture that distills patterns from representative research prototypes and industrial platforms [70, 86–88].

- **Perception and input encoding:** raw inputs (user queries, environment observations, logs) are normalized into a structured representation and combined with retrieved context via retrieval-augmented generation (RAG) pipelines [89, 90].
- **Memory subsystem:** short-term conversational history, long-term episodic trajectories, and semantic knowledge bases support context extension beyond the LLM window [91–93].
- **Reasoning and planning core:** a central LLM (or a small ensemble) performs chain-of-thought reasoning, task decomposition, high-level planning, and decision-making [9, 94, 95].
- **Tool interface layer:** the agent exposes tools as structured function calls, including search, databases, code execution, simulators, domain APIs, and other agents [10, 19, 87, 88, 96].
- **Actuation layer:** actions produced by the agent are executed in external environments (e.g., operating systems, code repositories, web browsers, robots, enterprise systems) via connectors and orchestrators [10, 45].Efficient deployment of LLM-based agents also depends on

hardware-aware optimizations such as heterogeneous edge accelerators[97].

- **Feedback and oversight:** internal reflection, critic models, human feedback, and monitoring services evaluate the agent's actions and update memory and policies [17, 18, 44, 92].

This modular view is agnostic to specific frameworks (e.g., AutoGen, LangChain, Semantic Kernel) and accommodates both single-agent and multi-agent designs [23, 87, 88]. In later sections we will show how concrete systems instantiate or partially omit different modules depending on their autonomy level and application domain.

Generic LLM-Agent Control Loop

The modular architecture can be instantiated as a generic control loop that alternates between perceiving new information, reasoning, acting (often via tools), and incorporating feedback. Algorithm 1 sketches a high-level pseudocode template that abstracts common patterns in ReAct-style agents [9], tool-using agents [19, 96], open-ended explorers [98], and workflow-based multi-agent systems [23, 26].

Algorithm 1 Generic LLM-agent control loop (single agent, potentially embedded in a multi-agent system).

Require: initial goal g, environment interface $\mathcal{E}$, tool set $\mathcal{T}$, memory $\mathcal{M}$, LLM policy f_θ
Initialize trajectory $\tau \leftarrow []$, internal state $s_0 \leftarrow \textsc{InitState}(g)$
for $t = 0, 1, 2, \ldots, T_{\max}$ **do**
  $o_t \leftarrow \textsc{Observe}(\mathcal{E}, s_t)$ {perception / new information}
  $c_t \leftarrow \textsc{BuildContext}(g, o_t, \mathcal{M}, \tau)$ {include retrieved memory}
  $a_t \leftarrow f_\theta(c_t)$ {LLM produces an action specification}
  if a_t is a tool call to some $T \in \mathcal{T}$ **then**
    $r_t \leftarrow \textsc{ExecuteTool}(T, a_t)$
  else
    $r_t \leftarrow \textsc{Act}(\mathcal{E}, a_t)$ {environment action or message}
  end if
  $\tau \leftarrow \tau \cup \{(o_t, a_t, r_t)\}$ {append to trajectory}
  $\mathcal{M} \leftarrow \textsc{UpdateMemory}(\mathcal{M}, o_t, a_t, r_t)$
  $(s_{t+1}, done) \leftarrow \textsc{UpdateState}(s_t, o_t, a_t, r_t)$
  $fb_t \leftarrow \textsc{Feedback}(g, \tau, \mathcal{M})$ {optional: critic or human feedback}
  if *done* **then**
    break
  end if
end for
return final state s_{t+1}, trajectory τ, memory $\mathcal{M}$

Concrete systems specialize this template in different ways. ReAct interleaves reasoning and acting by formatting c_t as a concatenation of "Thought:" and "Action:" segments [9]. Toolformer augments f_θ with tool tokens that are learned via self-supervision so that tool calls are embedded directly in the generated sequence [19]. Voyager extends the loop with a skill library that stores code snippets learned during exploration [98]. AutoGPT- and BabyAGI-style systems wrap the loop in outer planners and task queues to manage multiple goals over time [99, 100].

Four-Dimensional Taxonomy of LLM Agents

To compare heterogeneous systems within a unified space, we introduce a four-dimensional taxonomy with the following axes:

1) **Autonomy level** (A): degree to which the agent can operate without human intervention, considering horizon length, self-starting behavior, and self-improvement [7, 8, 98, 99].
2) **Tool awareness and integration** (T): richness of the tool interface and the sophistication with which tools are selected, composed, and created [10, 19, 87, 88, 96].
3) **Collaboration structure** (C): presence and topology of multi-agent interactions, including coordinator–worker, peer-to-peer, and hierarchical teams [23, 44, 84, 91].
4) **Safety and governance** (S): mechanisms for guardrails, monitoring, auditability, and organizational control over agent actions [70, 86, 87, 92, 101].

Each axis can be further discretized into levels. Table 4 proposes indicative levels that capture trends observed in current systems. These levels are not meant to be rigid categories; rather, they serve as a pragmatic scaffold for mapping systems and spotting under-explored regions of the design space.

Using these axes, a system can be roughly denoted by a tuple (A_i, T_j, C_k, S_ℓ). For example, a typical ReAct-style coding assistant with a human in the loop might be $(A1, T1, C0, S1)$, whereas an enterprise workflow agent orchestrating multiple APIs under strong monitoring could be $(A2, T2, C1, S2)$. A long-horizon open-world explorer with skill libraries and minimal safety controls might be $(A3, T2, C0, S0)$. Importantly, safety in agentic systems extends beyond content filtering to include control over actions, tool usage, and long-horizon behavior, introducing risks such as prompt injection, goal misalignment, and unintended side effects. Robustness against adversarial inputs remains a major concern in AI systems[102].

Operationalization of Taxonomy Coding

To ensure that the proposed four-dimensional taxonomy is applied consistently and reproducibly, we formalize the criteria used to map systems onto the autonomy (A), tool use (T), collaboration (C), and safety/governance (S) axes. Rather than relying on subjective interpretation, each dimension is

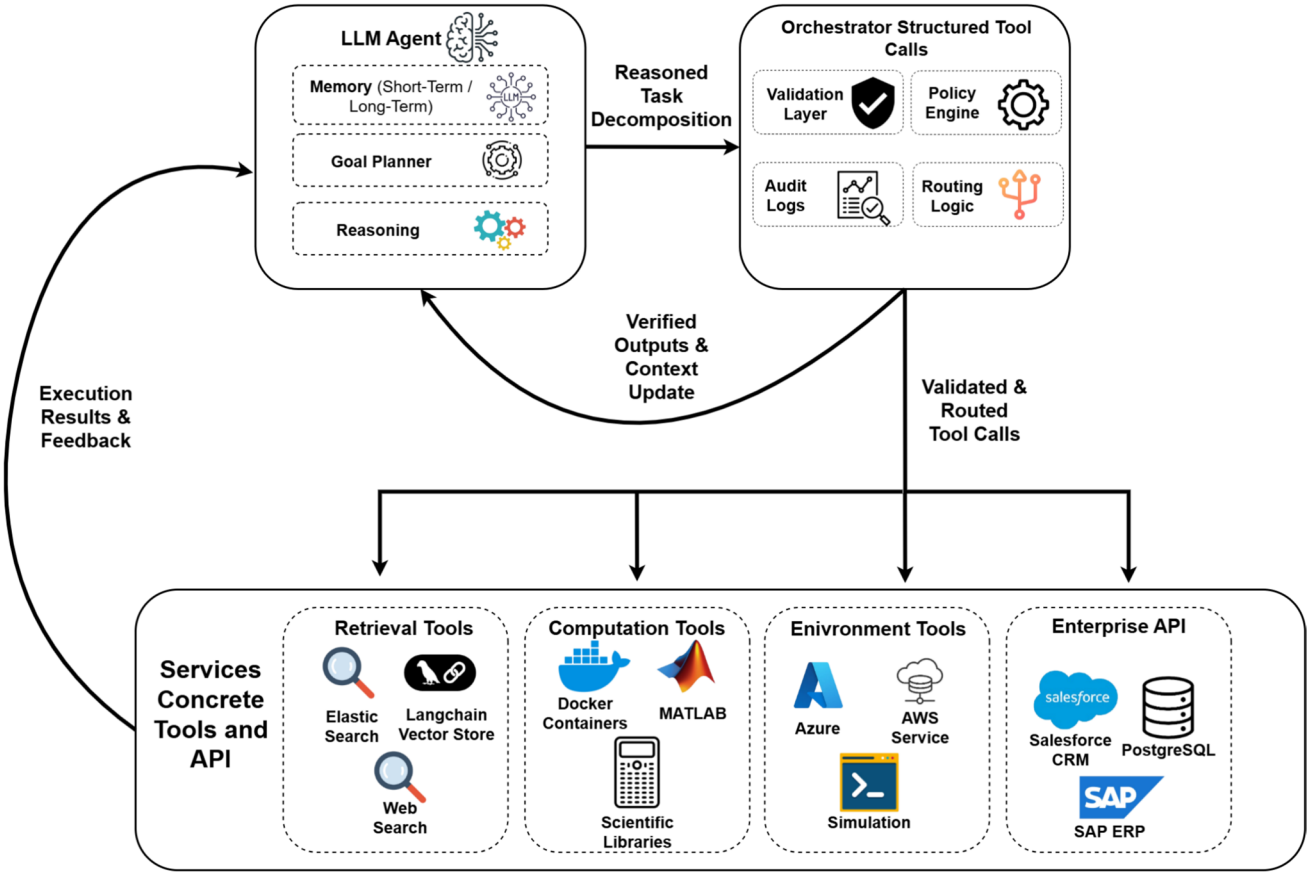


Fig. 3 Unified modular framework for LLM agents: encoded inputs and retrieved context are processed by an LLM-based reasoning and planning core that selects actions (tools, environment operations, or inter-agent messages), with memory and feedback updating future decisions

assigned based on observable system characteristics explicitly reported in the original papers or documentation.

Autonomy (A): Systems are categorized based on (i) horizon length (number of iterative steps), (ii) degree of human intervention, and (iii) ability to self-initiate or adapt tasks.

- A0: Single-turn or stateless interaction without iterative control.
- A1: Short-horizon multi-step reasoning with human guidance.
- A2: Long-horizon task execution with automated planning and limited human intervention.
- A3: Persistent or open-ended agents capable of self-improvement or continuous operation.**Tool Use (T):** Tool integration is classified based on the structure and sophistication of tool invocation:

- T0: No explicit tools beyond parametric knowledge.
- T1: Prompt-described or loosely structured tool invocation without guaranteed execution feedback (i.e., textual simulation of tool use)..
- T2: T2: Structured function calling with explicit execution feedback and integration into the agent's reasoning loop.
- T3: Tool synthesis or dynamic creation of reusable tools.**Collaboration (C):** Collaboration level is determined by the presence and structure of multi-agent interactions:

- C0: Single-agent system.
- C1: Implicit collaboration (shared memory/logs).
- C2: Explicit multi-agent protocols (e.g., planner–worker, debate).
- C3: Large-scale agent societies or simulations.**Safety and Governance (S):** Safety levels are assigned based on the presence of guardrails and oversight mechanisms:

- S0: No explicit safety mechanisms beyond base model.
- S1: Prompt-level safeguards or simple filters.
- S2: Centralized monitoring, logging, and policy enforcement.
- S3: Organization-level governance with compliance and risk controls.

Table 4 Indicative levels for the four-dimensional taxonomy of LLM agents

Axis	Level	Description
Autonomy (A)	A0	Single-turn assistant; no persistent goals or self-starting behavior.
Autonomy (A)	A1	Short episodic autonomy (few steps) with user in the loop.
Autonomy (A)	A2	Long-horizon task execution with automatic planning and recovery.
Autonomy (A)	A3	Lifelong operation with self-improvement and self-scheduling of goals [98, 99].
Tool use (T)	T0	No explicit tools; only parametric knowledge.
Tool use (T)	T1	Prompt-described tools; simple function calling.
Tool use (T)	T2	Model-native tool use with learned triggers and multi-tool composition [10, 19, 87].
Tool use (T)	T3	Tool-making agents that synthesize new tools or APIs for future reuse [93, 96].
Collaboration (C)	C0	Single agent without explicit collaboration.
Collaboration (C)	C1	Implicit collaboration via shared memory or logs.
Collaboration (C)	C2	Explicit multi-agent protocols (coordinator–worker, debate, voting) [23, 44].
Collaboration (C)	C3	Large-scale agent societies and simulations with emergent roles.
Safety / Gov. (S)	S0	No explicit safety mechanisms beyond base model filters.
Safety / Gov. (S)	S1	Prompt-based guardrails and ad hoc rule filters.
Safety / Gov. (S)	S2	Centralized policy engine, audit logs, and runtime monitors [45, 70].
Safety / Gov. (S)	S3	Organization-wide governance with risk modeling, approvals, and compliance integration [86, 101].

For each system, taxonomy labels are assigned by analyzing architectural descriptions, control loops, tool interfaces, and reported deployment constraints in the corresponding references. When ambiguity exists, we assign the lowest level consistent with documented evidence to avoid overestimating capabilities. These operationalized criteria form the basis for systematically mapping representative LLM agent systems to the proposed taxonomy, as summarized later in Table 5. To improve consistency, each taxonomy dimension is defined using observable system characteristics such as horizon length, degree of human intervention, structure of tool interfaces, and presence of explicit coordination protocols. These criteria are derived from recurring patterns reported in prior literature and are applied uniformly across all systems considered in this survey. While the resulting levels (A0–A3, T0–T3, C0–C3, S0–S3) are indicative rather than strictly quantitative, they provide a structured basis for comparative analysis.

Table 5 Illustrative mapping of representative LLM agent systems to taxonomy levels

System	Domain / type	A	T	C	S
ReAct [9]	Reasoning + acting agent	A1	T1	C0	S0
Toolformer [19]	Tool-using LM	A0	T2	C0	S0
HuggingGPT [10]	Model-routing agent	A1	T2	C1	S1
Voyager [98]	Open-world explorer	A3	T2	C0	S0
AutoGPT [99]	Goal-driven task executor	A2	T1	C0	S0
BabyAGI [100]	Iterative task planner	A2	T1	C0	S0
ToolMaker [96]	Tool-making planner	A1	T3	C0	S0
MemoryBank [93]	Memory-centric agent	A2	T2	C0	S1
AutoGen-style LLM-MAS [23]	Multi-agent framework	A1	T2	C2	S1
Workflow LLM-MAS [26]	Workflow orchestrator	A2	T2	C2	S1
Enterprise agent mesh [45, 70]	Enterprise workflows	A2	T2	C2	S2
Safe agent platforms [86, 87]	Managed tool-use platforms	A1	T2	C1	S2–S3

Defining the Boundary of Tool Integration

A key ambiguity in agentic systems is distinguishing between formatted text generation and substantive tool integration. While many systems can produce structured outputs (e.g., JSON, XML, or YAML) that are subsequently parsed by external scripts, not all such interactions constitute meaningful tool use from a systems perspective.

We define *tool integration* as the presence of a closed-loop interaction between the agent and external functions, where (i) tool invocation is explicitly represented in the control flow, (ii) execution results are programmatically returned to the agent, and (iii) subsequent reasoning steps depend on these results. Under this definition, merely emitting structured text for downstream parsing does not qualify as full tool integration unless the agent's behavior is conditioned on the tool's execution outcomes.

This distinction enables a clearer boundary across the tool-use spectrum:

- **Textual Tool Simulation:** The model emits structured text that is interpreted externally, without guaranteed feedback or integration into the reasoning loop.
- **Interface-Level Integration:** Tools are invoked via structured APIs (e.g., function calling), but control remains largely prompt-driven.
- **Controller-Orchestrated Integration:** Tool usage is embedded within an explicit execution loop, where results are validated, stored, and iteratively used for decision-making.

From an engineering standpoint, this boundary is critical: only the latter two categories provide reliable grounding and controllability, while purely text-based approaches remain fragile and difficult to verify.

Mapping Representative Systems to the Taxonomy

To make the taxonomy concrete, Table 5 maps representative systems and frameworks to the four axes.The mapping is based on the operationalized criteria defined in Section 3.4, ensuring that each assignment reflects explicitly documented system properties such as control loop structure, tool interface design, and collaboration protocols. While some degree of abstraction is unavoidable due to heterogeneous reporting across works, we adopt a conservative coding strategy that prioritizes verifiable evidence over inferred capabilities. [9, 10, 19, 23, 87, 88, 98–100].

To illustrate the application of these criteria, we briefly justify selected mappings. ReAct is categorized as (A1, T1, C0, S0) because it performs short-horizon iterative reasoning with prompt-defined tool usage and no explicit collaboration or safety mechanisms. Toolformer is assigned (A0, T2, C0, S0) as it integrates tools at the model level but does not operate within a multi-step control loop. Voyager is categorized as (A3, T2, C0, S0) due to its persistent operation, skill accumulation, and long-horizon exploration. Multi-agent frameworks such as AutoGen are labeled (A1, T2, C2, S1) because they involve structured inter-agent communication with moderate autonomy and basic safeguards.

These examples demonstrate how the taxonomy captures meaningful distinctions across systems while remaining grounded in observable architectural features. This mapping highlights several trends. First, many systems cluster around moderate autonomy (A1–A2) and moderate-to-advanced tool use (T1–T2), while high-autonomy, high-collaboration, strongly-governed systems ($A3$, $T2/T3$, $C2/C3$, $S3$) remain comparatively rare. Second, safety and governance levels tend to lag behind autonomy and tooling sophistication; research prototypes often exhibit high autonomy with minimal oversight, whereas industrial platforms trade autonomy for stronger control [45, 70, 101].

To assess the consistency of the taxonomy application, we performed an internal cross-verification of system mappings by independently reviewing a subset of representative agent systems and comparing assigned taxonomy levels based on the criteria defined in Section III-D. While a formal inter-rater reliability study is beyond the scope of this survey, this process helped ensure that the taxonomy assignments reflect consistent interpretation of observable system properties rather than ad hoc judgments.

Application Context and Engineering Utility

While the proposed taxonomy abstracts across autonomy, tool use, collaboration, and safety/governance, we acknowledge that real-world systems are also shaped by their application domains. Rather than introducing domain as an additional axis, we treat it as an orthogonal dimension that interacts with the taxonomy to guide system design decisions.Robotic manipulation in complex environments requires robust perception under occlusion [103].Accurate localization and motion estimation remain fundamental challenges in embodied agents [104]. Simulation environments are increasingly used to improve robustness under adverse conditions [105].Integration of multimodal spatial data improves localization accuracy in real-world environments [106]. From an engineering perspective, the taxonomy is intended to support design trade-offs rather than prescribe domain-specific architectures. For example, coding agents and embodied agents (e.g., robotics systems) differ significantly in their environments and constraints, yet both can be meaningfully compared along the A/T/C/S axes. A coding agent may operate at (A2, T2, C1, S1), emphasizing structured tool use and iterative reasoning, whereas a robotics agent may require lower autonomy but higher safety guarantees (e.g., A1, T2, C1, S3) due to physical-world risks.

This abstraction enables engineers to decouple *what the system does* (application domain) from *how it is architected* (agentic properties). In practice, the taxonomy can be used to:

- Identify appropriate levels of autonomy and control for a given task,
- Determine the complexity of tool integration required,
- Decide whether multi-agent collaboration is beneficial,
- Align safety mechanisms with system capabilities and risk exposure.

Thus, while domain-specific factors remain critical, the proposed taxonomy provides a consistent framework for reasoning about architectural trade-offs across heterogeneous applications.

Design Patterns and Trade-Offs

The framework and taxonomy also make it easier to discuss recurring design patterns and trade-offs:

Centralized Versus Distributed Control Single-agent architectures concentrate decision-making in one LLM core, simplifying reasoning but limiting robustness and specialization. Multi-agent systems distribute roles across planners, workers, critics, and tool proxies [23, 44], improving

modularity but requiring coordination protocols and conflict resolution.

Tight Versus Loose Tool Integration Tightly integrated tools (T2/T3) are triggered directly from model generations and may benefit from training-time alignment [19, 87], but they increase coupling between the model and environment APIs. Loosely coupled tools (T1) are orchestrated by an outer controller (e.g., Python code, workflow engines), improving debuggability at the cost of additional engineering effort [26, 88].

Explicit Versus Implicit Memory Some systems maintain explicit episodic and semantic memory stores [91, 93], while others rely on long context windows and prompt engineering. Explicit memory supports lifelong learning and cross-session personalization, but raises questions about storage cost, privacy, and retrieval quality.

Safety Baked-in Versus Safety Bolted-on Platforms that treat safety and governance as first-class design axes (S2–S3) embed guardrails, policy engines, and observability from the start [70, 86, 87]. By contrast, many research prototypes add safety layers post hoc, often limited to simple filters or manual review. As the field matures, we expect more systems to move safety and governance from an afterthought to a core component of the architecture.

The rest of this survey will repeatedly refer back to the modules in Fig. 3, the control loop in Algorithm 1, and the taxonomy levels in Tables 4 and 5 as we analyze autonomous agents (Section "Autonomous LLM Agents"), tool-using agents (Section 5), and LLM-based multi-agent systems (Section 6).

Design Implications Across the A/T/C/S Space

The unified framework and taxonomy suggest several practical design implications. First, higher autonomy levels (A2–A3) amplify the risks associated with tool misuse and error accumulation, necessitating stronger safety and governance mechanisms (S2–S3). Second, rich tool integration (T2–T3) improves capability and grounding but increases system complexity, making observability and auditability essential for deployment. Third, multi-agent collaboration (C2–C3) enhances robustness and parallelism, yet introduces coordination failures and emergent behaviors that are not captured by single-agent benchmarks. These observations indicate that autonomy, tool use, collaboration, and governance should be co-designed rather than optimized independently. Effective agentic systems therefore require architectural choices that balance capability gains against reliability, safety, and operational constraints.

Autonomous LLM Agents

Autonomous LLM agents extend simple assistants by embedding the model inside an explicit perception–action loop that can pursue goals over multiple steps with limited human intervention [7, 8]. They plan, act, monitor their progress, and often refine their own behavior using feedback and memory. This section reviews key design patterns for autonomy, including planning and task decomposition, self-reflection and feedback, memory-augmented acting, and long-horizon open-ended behavior. We also summarize representative systems across domains.

What Makes an LLM Agent "Autonomous"?

In classical AI, autonomy is often defined as the degree to which an agent can operate without direct external control, given a performance objective [107, 108]. For LLM agents, autonomy has several practical dimensions [7, 8, 40, 41]:

- **Initiation:** whether the agent can self-start tasks based on triggers or schedules, instead of waiting for a single prompt.
- **Horizon length:** the typical number of perception–action steps before termination (from a few to thousands).
- **Decision authority:** how often humans must review or approve actions.
- **Self-modification:** the extent to which the agent can update its own plans, skills, or memory over time.

Within our taxonomy (Section 3.3), these aspects determine the autonomy level $A \in \{A0, \ldots, A3\}$. While some works explicitly aim for "fully autonomous" agents [99, 100], production deployments typically choose intermediate autonomy levels and keep humans in the loop for high-risk decisions [45, 70, 101].

Despite these advances, autonomous agents remain brittle in long-horizon settings, where minor reasoning errors can cascade into complete task failure. Current approaches rely heavily on heuristic feedback mechanisms rather than principled guarantees of correctness, limiting their reliability in real-world deployment.

Planning and Task Decomposition

A central ingredient of autonomy is the ability to decompose complex goals into smaller sub-tasks. Early LLM work

Table 6 Representative planning patterns used in autonomous LLM agents

Pattern	Description	Example systems
Prompt-level sequential planning	Plan described in natural language; executed step by step by the same agent.	ReAct [9], Plan-and-Solve [109], AutoGPT [99]
Tree / graph search with LLM expansions	LLM proposes multiple next steps; external search explores branches and selects best.	Tree-of-Thoughts [110], Graph-of-Thoughts [111]
Planner–executor multi-agent pattern	Planner agent creates sub-tasks; workers or tools execute them.	Workflow LLM-MAS [26], HuggingGPT [10], code-agent frameworks [112]
Skill-library planning	Planner selects among learned skills or tools stored in a library.	Voyager [98], MemoryBank[93]

showed that chain-of-thought prompting can reveal latent reasoning capabilities [95], but autonomous agents require more structured planning (Table 6).

Prompt-Level Planners Simple agents implement planning entirely at the prompt level: they ask the LLM to "think step by step", produce a list of subgoals, and then iterate over them. ReAct [9] interleaves "Thought" and "Action" steps, thereby blending planning and acting. Plan-and-solve variants first generate a high-level plan, then execute each step potentially with new prompts and tools [94, 109].

Tree- and Graph-Structured Search More advanced systems treat the LLM as a node-expansion oracle inside a search procedure. Tree-of-Thoughts explores multiple reasoning branches and uses voting or heuristics to pick promising paths [110]. Graph-of-Thoughts generalizes this to graph structures where intermediate states and sub-solutions can be reused [111]. These approaches trade more computation for improved global planning and robustness.

Planner–Executor Architectures Other systems separate planning and execution into distinct agents. A planner agent decomposes the goal into tasks, while executor agents call tools or operate in environments [10, 23]. This pattern is common in code-generation workflows and multi-step business processes, as it mirrors human organization structures [86, 112].

Self-Reflection and Feedback-Driven Improvement

Even sophisticated planners make mistakes. Autonomous agents therefore rely heavily on feedback, both internal and external, to detect and correct errors [17, 18, 92] (Table 7).

Self-reflection Self-reflection methods ask the same model (or a copy of it) to critique its previous outputs and suggest

Table 7 Examples of feedback mechanisms in autonomous LLM agents

Feedback type	Mechanism	Representative works
Internal self-reflection	Agent prompts itself to critique and revise previous outputs.	Self-Refine [18], Reflexion [17]
External model feedback	Separate critic model scores or comments on actions or plans.	Feedback survey [92], debate-style systems [44, 113]
Human-in-the-loop	Human reviewers approve or adjust agent actions, especially for high-risk steps.	RLHF [83], enterprise agents [45, 70]
Environment rewards	Rewards from environment used for exploration and skill learning.	Voyager [98], game-playing agents [114, 115]

revisions [17, 18]. For example, reflexion-style agents maintain a memory of failures and surface them as hints in future prompts [17]. Empirically, such mechanisms improve success rates on coding, math, and instruction-following benchmarks by enabling iterative repair.

External Model and Human Feedback Other systems use a dedicated critic model or human annotators to evaluate trajectories. Feedback can be integrated at inference time (e.g., reject-sampling, ranking) or used to fine-tune policies via reinforcement learning from human feedback (RLHF) and its variants [29, 83].

Recent improvements in reinforcement learning from human feedback focus on cooperative optimization between policy and reward models.[116].

Agents operating in high-stakes domains (finance, healthcare, legal) often combine internal reflection with human approval for critical actions [45, 70].

Environment and Reward Feedback In simulated or game-like environments, reward signals from the environment guide exploration and policy improvement [98, 114, 115]. For LLM agents, this often takes the form of task-specific success metrics (e.g., game score, task completion flags) or shaped rewards based on intermediate achievements.

Memory-Augmented Autonomy

As agents operate over longer horizons, they must maintain and exploit memory. Zhang et al. survey memory mechanisms for LLM-based agents, categorizing them into short-term, long-term episodic, and semantic memories [91]. Systems such as MemoryBank provide explicit memory modules that store key-value pairs representing experiences; these are

retrieved and injected into prompts when relevant [93]. Generative agents in simulated social environments use memory streams to construct higher-level reflections and plans [24].

Memory interacts tightly with autonomy:

- At low autonomy levels (A1), agents often rely on the context window plus a small scratchpad history.
- At moderate levels (A2), agents use vector databases or key-value stores to remember tasks, tools, and prior decisions [88, 91, 93].
- At high levels (A3), agents maintain lifelong stores of skills and episodic trajectories, which can be mined for meta-learning and reflection [24, 98].

Memory design must balance capacity, retrieval accuracy, and privacy. Storing every interaction is costly and may leak sensitive information; selective summarization and forgetting become necessary as agents scale [40, 91].

Long-Horizon and Open-Ended Autonomy

Long-horizon settings, such as open-world games, social simulations, and persistent enterprise workflows, stress-test autonomy. Voyager operates in Minecraft indefinitely, learning a growing library of executable skills and using code generation to interact with the environment [98]. Park et al.'s *Generative Agents* simulate dozens of agents living in a sandbox town, forming habits, planning for events, and coordinating social activities [24]. Enterprise agent meshes orchestrate long-running workflows across CRM, ERP, and monitoring systems [45, 70].

These settings expose several challenges:

- **Error accumulation:** small reasoning or perception errors can compound over long trajectories.
- **Non-stationarity:** environments and tools change over time, invalidating learned patterns.
- **Safety drift:** agents that update their own memory or skills may gradually deviate from initial safety constraints.

Addressing these issues likely requires hybrid approaches combining LLM-based planning with classical control, formal verification, and robust monitoring [40, 44, 45].

Application Domains and Case Studies

Autonomous LLM agents have been explored in diverse domains:

Software Engineering and DevOps Code agents automate bug triaging, patch generation, test synthesis, and CI/CD workflows [112, 117, 118]. They typically operate at autonomy levels A1–A2, with human approval for patches and deployments.

Web and Tool Navigation Web-browsing agents use tools such as WebArena and related environments to complete tasks like booking travel or filling forms [22, 119]. These systems highlight the difficulty of perception and robustness in noisy, real-world web interfaces.

Scientific Discovery and Analysis Autonomous agents assist with literature review, hypothesis generation, and experimental design [120, 121]. They act as collaborators that can propose experiments, search literature, and maintain research notebooks over time.

Social Simulations and Education Generative social agents simulate classrooms, workplaces, or societies to study emergent behavior or provide training environments [24, 122]. Educational agents autonomously customize lesson plans and interactive exercises for students over extended periods [123].

Across these domains, a recurring pattern is that autonomy is carefully scoped: agents are given clear boundaries (tools, domains, policies), and humans retain veto power over high-impact actions [45, 70, 101].

Summary and Open Questions for Autonomy

Autonomous LLM agents bring the promise of continuous, multi-step problem-solving, but they also surface unresolved questions:

- How can we systematically choose an autonomy level $(A0–A3)$ that matches risk, domain, and user needs?
- What principled methods exist for combining LLM-based planners with classical search and control to improve reliability?
- How should we design memory and feedback loops so that agents improve over time without accumulating unsafe behaviors?

The next sections address two closely related axes of our taxonomy: tool-using agents (Section 5) and multi-agent collaboration (Section 6), both of which interact strongly with autonomy.

Tool-Using and Tool-Making LLM Agents

While autonomy focuses on *how long* an agent can operate without intervention, tool use determines *how far* its capabilities extend beyond text generation. Tool-using LLM agents ground their outputs in external data sources, calculators, code execution environments, simulators, and enterprise systems [9, 10, 19, 87–90]. Recent work further explores *tool-making* agents, where LLMs synthesize new tools or APIs that can be reused by themselves or other agents [93, 96]. This section characterizes tool abstractions, integration patterns, and the emerging landscape of tool-making agents, with an emphasis on reliability and security.

What Counts as a "Tool"?

In this survey, a *tool* is any external capability that the agent can invoke through a structured interface (e.g., function call, REST API, RPC) to obtain information or perform actions beyond pure text generation [19, 87, 88]. Typical tool classes include:

- **Information retrieval:** web search, document retrieval, vector stores, database queries [89, 90, 124, 125].
- **Computation and code execution:** calculators, symbolic algebra systems, Python/Julia sandboxes, spreadsheet APIs [5, 10, 90].
- **Environment interaction:** browser automation, file systems, operating system commands, simulation environments, robotics middleware [10, 22, 98, 119].
- **Domain-specific services:** CRM/ERP APIs, financial trading endpoints, medical information systems, code hosting platforms [45, 70, 86].
- **Meta-tools:** other LLMs, critic models, or orchestration services used as subroutines [23, 44].

Table 8 summarizes common tool categories and their roles within agent workflows.

Figure 4 shows a simplified stack that distinguishes between the *tool broker* (which exposes a catalog of tools and handles authentication, rate limiting, and logging) and the underlying services.

Table 8 Illustrative categories of tools used by LLM agents

Tool category	Typical capabilities	Example usage
Retrieval tools	Web search, vector DB, SQL/NoSQL queries	Ground answers in recent or proprietary data [89, 90, 124]
Compute tools	Calculators, code sandboxes, spreadsheets	Offload arithmetic, simulation, or batch data processing [5, 10]
Environment tools	Browsers, file systems, game engines, robots	Navigate websites, edit files, control agents in simulations [22, 98, 119]
Enterprise tools	Business apps, knowledge systems, ticketing APIs	Execute workflow steps in CRM/ERP/ITSM systems [45, 70, 86]
Meta-tools	Other LLMs, critics, orchestrators	Obtain second opinions, debate, or specialized reasoning [23, 44, 92]

Interfaces and Integration Patterns for Tools

Modern tool-using agents largely rely on three interface styles.

Prompt-Described Tools Early systems described tools purely in natural language within the prompt. For instance, ReAct instructs the model that it can use a "Search" tool and asks it to explicitly output tool invocations as textual actions [9]. This style is simple but brittle: the model must learn the API syntax implicitly and cannot rely on strong type checking.

Structured Function Calling APIs such as OpenAI function calling and similar mechanisms in Claude and Gemini expose tools as JSON schemas with explicit argument types [5, 86, 87]. The LLM produces a JSON-compatible object that is validated and then executed by the runtime. Frameworks like LangChain, Semantic Kernel, and AutoGen wrap APIs, databases, and custom functions into reusable tools that can be called from prompts or higher-level controllers [23, 26, 88].

Controller-Orchestrated Tools In many production systems, a separate controller interprets the LLM's outputs (e.g., XML or special markers) and manages tool calls explicitly [10, 45, 70]. This approach sacrifices some immediacy but gains debuggability, logging, and the option to insert approval or policy checks before executing tools.

These interfaces can be combined: a tool call may first be drafted in natural language, then parsed into a structured representation, and finally inspected by a controller. Figure 5 sketches a typical function-calling workflow.

Patterns of Tool use in LLM Agents

Empirically, tool use in LLM agents tends to follow a small number of patterns.

Single-Hop Grounding The agent uses a retrieval or compute tool once to answer the current query, without maintaining state across turns. Retrieval-augmented generation

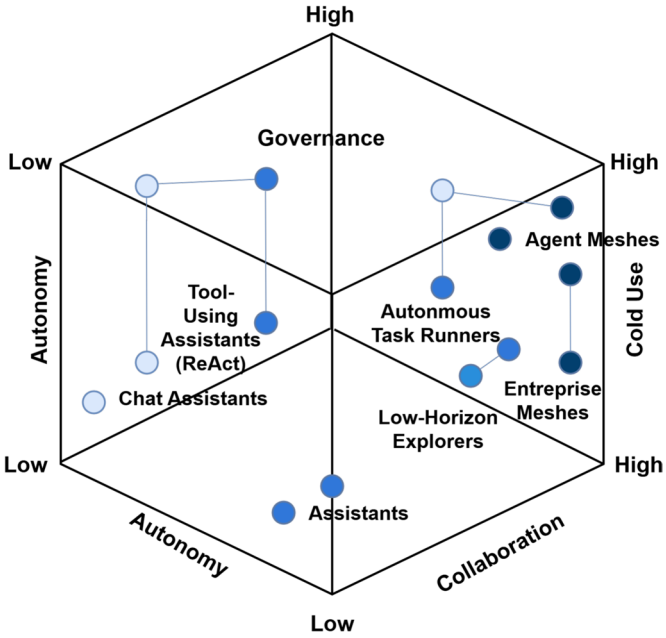


Fig. 4 Conceptual spectrum of autonomy for LLM agents, ranging from short-horizon assistants under full human supervision to long-horizon systems with mixed human and automated control

for QA and summarization often follows this pattern [89, 90, 125, 126].

Multi-Hop Tool Chains The agent constructs short chains of tools, such as search → parse → aggregate, or data extraction → transformation → visualization [10, 88]. Workflow-based systems expose these chains as reusable templates that can be invoked by users or other agents.

Reactive Planning with Tools ReAct-style agents interleave thoughts and actions, choosing at each step whether to call a tool or continue reasoning in-context [9, 110]. This flexible pattern suits tasks where the agent must explore, check intermediate hypotheses, or backtrack.

Tool-Centric Orchestration Some frameworks treat tools as the primary units and use LLMs mainly for routing between them. HuggingGPT, for example, uses an LLM to assign subtasks in a multi-modal pipeline to specialized models hosted on Hugging Face [10]. In enterprise settings, graphs of tools may be orchestrated by workflow engines with LLMs acting as decision points [45, 70].

Table 9 aligns these patterns with taxonomy levels on the tool axis.

Tool-Making Agents

Beyond calling existing tools, recent work explores *tool-making* agents that generate new functions, APIs, or workflows.

Toolformer showed that LLMs can learn to insert tool calls into text via self-supervised training on synthetic annotations [19]. Cai et al. extend this idea by asking LLMs to create new tools—small code snippets or API wrappers—that are then registered in a catalog and can be invoked later [96]. MemoryBank similarly allows agents to accumulate reusable skills in the form of parameterized code templates [93]. Other systems such as Voyager and various code-agent frameworks treat code repositories themselves as evolving toolboxes; agents write new programs that future episodes can call as tools [98, 112, 117].

Tool-making raises additional challenges:

- **Validation and testing:** synthesized tools must be checked for correctness, performance, and security before being added to the catalog.
- **Versioning and provenance:** agents need to track which version of a tool was used to produce which results.
- **Governance:** organizations must decide which agents are allowed to create tools and under what review processes [45, 70].

Reliability, Grounding, and Robustness

Tool use is supposed to make agents more reliable by grounding them in external systems, but it also introduces new failure modes:

Hallucinated Tool Calls and Arguments Agents may hallucinate APIs, endpoints, or parameters that do not exist [19, 21].

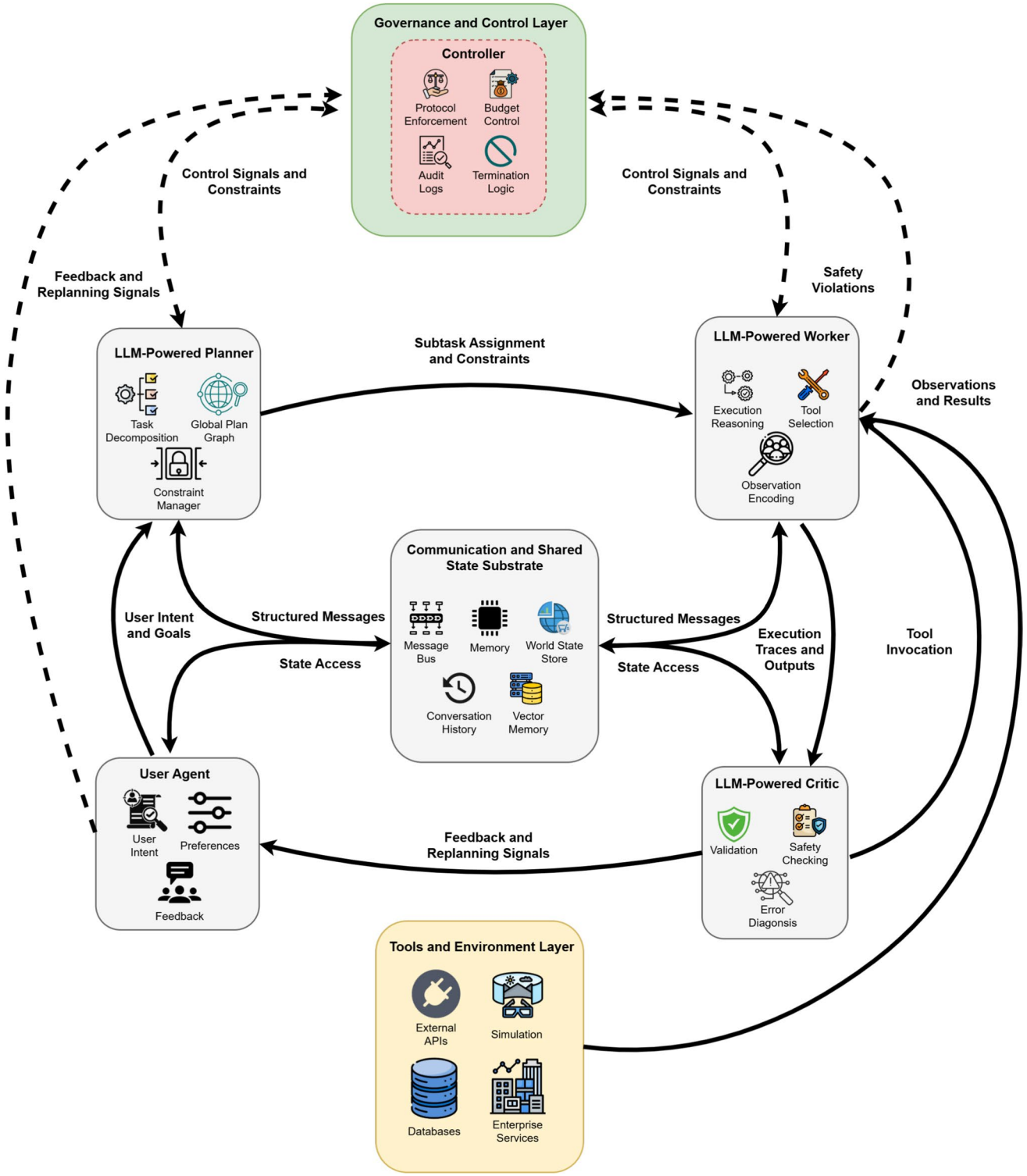


Fig. 5 Typical function-calling workflow for tool-using agents, where the LLM proposes a structured call, a controller validates and executes the tool, and results are fed back to produce grounded outputs

Table 9 Tool-use patterns and their relation to tool-integration levels

Tool-use pattern	Characteristics	Typical T-level
Single-hop grounding	One-off retrieval or compute call; minimal state.	T1–T2 [89, 90]
Multi-hop tool chains	Short pipelines of tools; often templated as workflows.	T2 [10, 26]
Reactive planning with tools	Interleaved reasoning and acting; dynamic choice of tools at each step.	T1–T2 [9, 110]
Tool-centric orchestration	LLM routes among many specialized tools or models; heavy controller involvement.	T2 [10, 45]
Tool-making and reuse	Agent synthesizes new tools and stores them for future calls.	T3 [93, 96]

Models trained on limited examples may overfit to spurious tool call formats or misuse tools in subtle ways.

Partial or Outdated Grounding Retrieval tools can return stale or irrelevant documents; database schemas may change over time. Robust tool-using agents incorporate schema discovery, health checks, and fallbacks when tools fail or return low-quality data [45, 90, 125].

Latency and Cost Each tool call incurs latency and potential monetary cost (e.g., external APIs, compute-heavy simulations). Agents must trade off additional tools against user experience and budget constraints [10, 70, 86]. Scheduling and caching strategies become important as tool catalogs grow [45].

Evaluation frameworks are beginning to measure these aspects explicitly, for example by tracking success rates conditioned on tool availability, tool latency distributions, and rates of invalid tool calls [19, 21, 90].

Security, Privacy, and Governance for Tool use

Because tools can read and modify external systems, misspecified or compromised tool use can have significant consequences [45, 70, 87, 101]. Security and governance mechanisms typically operate at three layers.

Tool Design and Scoping Tools should implement the principle of least privilege: access should be scoped to particular resources and actions (e.g., "read-only analytics" vs. "execute trades"). Static allow-lists and deny-lists limit which tools can be used in which contexts [86, 87].

Runtime Policies and Monitoring Policy engines check tool calls against business rules (e.g., threshold limits, restricted customers) before execution. Runtime monitoring logs all tool usage for auditing and incident response [45, 70]. Some platforms support "shadow" or dry-run modes where agents propose tool calls that are logged but not executed until approved.

User-in-the-Loop Control Interfaces can require human confirmation for sensitive tools (e.g., financial transfers, code deployment), sometimes with model-generated rationales to support review [70, 101]. This creates hybrid workflows where agents handle routine steps autonomously and escalate decisions that exceed predefined risk thresholds. These risks are further amplified in agentic settings, where compromised tool interactions can propagate across multiple steps, making traditional input-output safety checks insufficient. Overall, effective tool governance is a key determinant of whether enterprises are willing to deploy LLM agents beyond prototypes [45, 70, 101].

Summary and Link to Multi-Agent Collaboration

Tool-using and tool-making agents expand the reach of LLMs from pure text processing to acting on the world. Our taxonomy captures this expansion along the tool axis (T0–T3), from simple retrieval calls to agents that synthesize and manage their own tools. However, many real-world systems combine rich tool use with *multi-agent* collaboration, where different agents specialize in planning, acting, critiquing, or interfacing with specific tool families [23, 44]. The next section turns to these multi-agent architectures and examines how collaboration structures interact with autonomy and tool use.

LLM-Based Multi-Agent Systems and Collaboration

To maintain balance with earlier sections, we extend this discussion with a more detailed analysis of coordination mechanisms, failure modes, and scalability challenges specific to multi-agent settings. Thus far we have focused on single LLM agents. However, many recent systems organize *multiple* LLM-based agents that interact with each other and with humans to solve complex tasks [23, 44, 84]. These LLM-based multi-agent systems (LLM-MAS) leverage specialization, division of labor, and social dynamics such as debate and negotiation. This section reviews core architectures, communication patterns, collaboration protocols, emergent phenomena, and application domains.

From Single Agents to Multi-Agent Organizations

Classical multi-agent systems organize agents into teams, coalitions, and societies with explicit coordination

mechanisms [108, 127]. LLM-MAS inherit many of these ideas, but replace hand-coded agent logic with LLM-based policies that communicate in natural language. Surveys by Guo et al., Li et al., Chen et al., and Zhang et al. provide comprehensive overviews of LLM-MAS architectures, communication schemes, and applications [23, 44, 84].

A typical LLM-MAS consists of:

- A set of **agents** with roles (e.g., planner, coder, critic, tester, user proxy).
- A **communication substrate** (shared memory, message bus, or environment) through which agents exchange messages.
- An optional **coordinator** or **environment simulator** that enforces protocols and terminates conversations.

Role Design and Organizational Structures

LLM-MAS typically differentiate agents by *roles*, each with a dedicated system prompt, tools, and objectives. CAMEL introduces cooperative role-playing between two agents (e.g., "AI assistant" and "user") to explore task-solving dialogs [38]. MetaGPT treats software development as a multi-agent organization with roles such as product manager, architect, engineer, and QA, coordinated via a project-style workflow [128]. ChatDev uses communicative agents to simulate a software company that designs, implements, and tests software projects end-to-end [129].

Organizational structures can be roughly grouped as in Table 10.

Role design is partly an engineering art: prompts must balance specificity (to encourage specialization) with flexibility (to avoid brittle behavior). Frameworks such as AgentVerse and ChatArena provide libraries of reusable role templates and environments for experimentation [85, 130].

Table 10 Common organizational structures in LLM-based multi-agent systems

Structure	Characteristics	Representative systems
Coordinator–worker	Central coordinator assigns tasks to worker agents, aggregates results.	AutoGen [39], Workflow LLM-MAS [26], MetaGPT [128]
Peer-to-peer team	Symmetric agents discuss, debate, or vote without strict central control.	CAMEL [38], ChatArena [130], AgentVerse [85]
Hierarchical organization	Multiple layers (e.g., manager, team leads, workers) mirroring human orgs.	MetaGPT [128], ChatDev [129], enterprise meshes [45]
Environment-centric society	Agents interact primarily via a simulated world, not explicit messages.	Generative Agents [24], Sotopia [131], social simulators [122]

Communication Patterns and Protocols

Communication is the core mechanism through which LLM agents coordinate. Most systems use natural-language messages with light structuring (e.g., message types, metadata), although some introduce more formal protocols.

Turn-Taking Dialogs Many two-agent setups (e.g., CAMEL, AutoGen, simple planner–worker systems) use strict turn-taking: agents alternate generating messages until a termination condition is met [38, 39]. This pattern is easy to implement and debug, but may limit scalability as the number of agents grows.

Broadcast and Moderated Channels In larger teams, messages may be broadcast to all agents or routed through a moderator. AgentVerse and ChatArena support broadcast-style communication where agents read from a shared message history and decide whether to speak [85, 130]. Moderators can prune, summarize, or reorder messages to control context length.

Conversation Games and Negotiation Protocols Multi-agent debate, auctions, and negotiation games instantiate explicit protocols over language [44, 130, 131]. For example, debate-style systems encourage agents with different perspectives to argue, then use a judge agent or voting mechanism to select an answer [44, 113].

Shared Memory and Blackboard Architectures Instead of direct messaging, some systems rely on a shared memory or blackboard. Agents read from and write to this memory, which acts as both communication channel and long-term state [24, 91, 93]. This style is common in social simulations and persistent environments.

Communication patterns directly influence the collaboration level C in our taxonomy (Section 3.3): from simple C1 shared memory to fully explicit C2/C3 multi-agent protocols.

Collaboration, Competition, and Emergent Behavior

LLM-MAS have been used to study emergent social behaviors such as cooperation, norm formation, and deception [24, 44, 122, 131]. Sotopia, for example, simulates social dilemmas (e.g., prisoner's dilemma, public goods games) with LLM agents and observes when they choose to cooperate

or defect [131]. Generative Agents exhibit emergent patterns such as organizing parties and spreading information through gossip [24] (Table 11).

These studies highlight both opportunities and concerns:

- **Collective intelligence:** groups of agents can outperform single agents through specialization, critique, and voting [23, 44, 113].
- **Coordination failures:** without appropriate protocols, agents may talk past each other, repeat work, or get stuck in loops [23, 26, 39].
- **Emergent misbehavior:** agents can collude to break rules, propagate misinformation, or exploit poorly designed incentives [40, 44, 131].

Understanding and controlling emergent behavior is an active research frontier at the intersection of AI safety, game theory, and computational social science.

Evaluation of Multi-Agent Systems

Evaluating LLM-MAS is more challenging than evaluating single agents because outcomes depend on interactions among agents and the environment [23, 44, 84]. Common evaluation axes include:

- **Task performance:** success rates on collaborative tasks such as software development, web navigation, or game playing [22, 128, 129].
- **Process metrics:** number of turns, message lengths, redundancy, and tool usage patterns [26, 39, 85].
- **Social metrics:** measures of cooperation, fairness, polarization, norm adherence, or diversity of viewpoints in simulations [44, 122, 131].

Table 11 Representative LLM-based multi-agent frameworks and their characteristics

Framework	Focus / domain	Notable features
CAMEL [38]	Two-agent cooperation	Role-playing templates, automatic conversation generation
AutoGen [39]	General multi-agent apps	Flexible agent graph, human-in-the-loop integration
AgentVerse [85]	Multi-agent simulations	Environment abstractions, reusable role library
MetaGPT [128]	Software engineering	Organization-style workflow, role hierarchy
ChatDev [129]	Software company simulation	Full SDLC pipeline with communicative agents
ChatArena [130]	Multi-agent evaluation	Competitive/cooperative games, multi-backend support
Sotopia [131]	Social dilemmas	Environment for studying cooperation and norms
Generative Agents [24]	Social simulation	Rich memory and reflection over agent lives

- **Robustness and stability:** sensitivity to prompt variations, role changes, or perturbations in the environment [44, 130].

Benchmarks such as ChatArena, AgentVerse, and Sotopia provide standardized environments for comparing multi-agent setups under different prompts and model backends [85, 130, 131].

While multi-agent collaboration enhances modularity and diversity of reasoning, it also introduces coordination overhead and instability. Empirical studies show that without carefully designed protocols, agents may exhibit redundant communication, fail to converge, or reinforce incorrect reasoning through consensus.

Applications of LLM-MAS

LLM-MAS have been applied to a variety of domains beyond social simulation:

Software and Product Development Systems like MetaGPT and ChatDev model a full software company, assigning tasks such as requirement analysis, design, implementation, and testing to different agents [112, 128, 129]. Multi-agent code assistants can parallelize exploration of design options or alternative patches [112, 117].

Complex Decision Support Multi-agent debate and committee-style systems aim to improve decision quality in domains such as policy analysis, legal reasoning, and strategic planning [40, 44, 113]. Agents with different objectives (e.g., optimistic vs. pessimistic) can highlight trade-offs and failure modes.

Education and Training Multi-agent classrooms and training scenarios simulate peers, instructors, and evaluators, enabling realistic practice settings for learners [24, 122, 123]. Teachers can orchestrate agent teams to generate diverse feedback on student work.

Enterprise Workflows In enterprise settings, multi-agent meshes can separate concerns between policy enforcement, domain-specific tools, and user-facing copilots [45, 70, 86]. For instance, a planner agent may coordinate a compliance agent, a data-access agent, and a reporting agent to fulfill business requests under governance constraints.

Summary and Research Directions for Collaboration

LLM-based multi-agent systems move from "one powerful assistant" to *organizations* of agents that collaborate and sometimes compete. They provide a natural way to combine

specialization, tool ecosystems, and social reasoning, but also magnify risks related to coordination, safety, and emergent behavior.

Open research questions include:

- How can we design communication protocols and incentives that reliably yield beneficial collaboration without unintended collusion or conflict?
- What abstractions best capture large-scale agent societies (C3) while remaining controllable and auditable?
- How should evaluation frameworks capture both task performance and social properties such as fairness, robustness, and long-term stability?

Answering these questions is crucial for safely deploying collaborative LLM agents in high-impact domains such as governance, finance, and critical infrastructure.

Cross-Cutting Failure Modes and Limitations

While existing literature demonstrates rapid progress in LLM-based agents, a critical examination reveals recurring failure modes that cut across autonomy, tool use, and multi-agent collaboration. These limitations highlight gaps between prototype capabilities and reliable real-world deployment.

1) Error Accumulation in Long-Horizon Tasks: Autonomous agents operating at higher autonomy levels (A2–A3) are particularly susceptible to compounding errors. Small inaccuracies in early planning or tool outputs can propagate across steps, leading to task failure. Systems such as AutoGPT and Voyager illustrate that increased autonomy often comes at the cost of reduced reliability in extended trajectories.

2) Tool Misuse and Fragile Grounding: Although tool integration improves factual grounding, agents frequently misuse tools due to hallucinated API calls, incorrect argument formatting, or misinterpretation of tool outputs. Even structured function-calling interfaces (T2) do not fully eliminate these issues, especially in dynamic environments where APIs evolve.

3) Planning–Execution Mismatch: Many agents generate high-level plans that are not executable in practice. This mismatch arises because LLMs optimize for plausible reasoning rather than operational feasibility, leading to plans that omit constraints such as resource limits, tool availability, or environment dynamics.

4) Coordination Failures in Multi-Agent Systems: In LLM-MAS settings, agents may exhibit redundant communication, conflicting actions, or failure to converge. Without carefully designed protocols, systems can enter infinite loops or produce inconsistent outputs. This challenge becomes more pronounced at higher collaboration levels (C2–C3).

5) Lack of Robust Safety and Governance: Safety mechanisms often lag behind increases in autonomy and tool access. Many research prototypes operate at (A2–A3, T2) while remaining at S0–S1, exposing risks such as unauthorized actions, data leakage, or policy violations. This imbalance limits deployment in safety-critical domains.

6) Cost and Latency Trade-offs: Advanced agent behaviors—such as multi-step reasoning, tool chaining, and multi-agent debate—incur significant computational and monetary costs. In practice, these overheads constrain scalability and necessitate trade-offs between performance and efficiency.

7) Limited Generalization Across Domains: Many systems are evaluated in narrow environments (e.g., coding benchmarks, simulated worlds) and fail to generalize to real-world settings with noisy inputs, incomplete information, and changing constraints.

These failure modes suggest that improving agentic AI requires not only scaling models and tools but also developing principled methods for reliability, coordination, and governance. Importantly, these limitations motivate the need for stronger evaluation frameworks and deployment-aware design, as discussed in the following section.

Evaluation and Benchmarks for LLM Agents

We further expand this section to provide a more structured synthesis of evaluation challenges, including limitations of existing benchmarks and the need for agent-specific metrics. Importantly, evaluation requirements depend strongly on where an agent lies in the A/T/C/S design space. For example, high-autonomy agents require long-horizon robustness metrics, tool-intensive agents require grounding and failure analysis, and multi-agent systems require coordination and social metrics. Traditional NLP benchmarks focus on *one-shot* input–output behavior, whereas agents operate over trajectories, tools, and interactions with other agents and humans [30, 82]. In this section we review evaluation dimensions, benchmarks for single and multi-agent settings, and emerging methodologies for assessing safety, robustness, and cost.

Evaluation Dimensions

Following recent surveys and evaluation frameworks [23, 30, 44, 90, 132], we distinguish several complementary dimensions:

- **Task performance:** success or accuracy on well-defined tasks (e.g., pass@k for coding, success rate for web navigation, win rate in games).
- **Process quality:** properties of the trajectory such as number of steps, tool calls, backtracking behavior, and adherence to protocols.
- **Grounding and factuality:** correctness of claimed facts relative to trusted sources [125, 133, 134].
- **Safety and alignment:** avoidance of harmful, biased, or policy-violating behavior [29, 42].
- **Cost and efficiency:** latency, token usage, tool-call overhead, and monetary cost per successful task [10, 30, 45].
- **Human experience:** user satisfaction, trust, and cognitive load, often measured via human studies or preference modeling [24, 29, 83].

Table 12 summarizes these dimensions and typical metrics.

From Static LM Benchmarks to Agent Benchmarks

Most LLMs are initially evaluated on static benchmarks such as MMLU, BIG-Bench [82], GSM8K [136], HumanEval [135], and general-purpose suites like HELM [30]. While informative, these benchmarks primarily test conditional generation on fixed inputs.

Agentic evaluation requires moving from single-step tasks to *trajectories* in environments. AgentBench explicitly evaluates LLMs as agents over a variety of environments including web navigation, embodied tasks, and games [27]. WebArena and Mind2Web provide realistic web environments for browsing, form-filling, and interaction [22, 137]. ALFWorld and BabyAI offer textual or grid-based environments for instruction-following agents [138, 139].

Table 12 Key evaluation dimensions and typical metrics for LLM agents

Dimension	Representative metrics	Example references
Task performance	Accuracy, success rate, pass@k, win rate, completion time	HELM [30], BIG-Bench [82], MMLU, HumanEval [135]
Process quality	#steps, #tool calls, redundancy, protocol violations	AgentBench [27], AutoGen [39], Workflow LLM-MAS [26]
Grounding / factuality	Fact accuracy, hallucination rate, citation correctness	TruthfulQA [133], WebGPT, RAG evals [89, 90]
Safety / alignment	Policy violation rate, red-team success, toxicity scores	RLHF [83], Constitutional AI [29], safety surveys [42]
Cost / efficiency	Latency, tokens per success, tool cost per task	HELM [30], HuggingGPT [10], enterprise agents [45]
Human experience	Preference scores, usability ratings, trust measures	RLHF [83], user studies [24, 122]

Figure 6 conceptually illustrates the evaluation pipeline for agents: generating trajectories under a policy, logging actions and tool calls, computing task and process metrics, and optionally collecting human feedback.

Benchmarks for Single and Tool-using Agents

Several benchmarks target single or tool-using agents specifically.

Web and Tool Interaction WebArena and Mind2Web evaluate the ability of agents to complete web tasks such as booking tickets or updating dashboards [22, 137]. GAIA proposes a benchmark for general AI assistants that require tool use and multi-step reasoning across open-domain tasks [140]. ToolBench and Gorilla-style benchmarks focus on calling real-world APIs correctly and robustly [21, 141].

Code and Software Tasks HumanEval and related code benchmarks measure program synthesis, but agentic code assistants must also manage files, tests, and tools such as compilers and linters [112, 117, 135]. Recent work proposes agent-centric evaluations where agents iteratively run tests, inspect error messages, and patch code until success or timeout [117, 118].

Instruction-Following and Planning Benchmarks such as BabyAI, ALFWorld, and TextWorld test instruction-following and planning in partially observable environments [138, 139, 142]. AgentBench aggregates several of these and adds new scenarios tailored to LLM agents [27].

Table 13 lists representative benchmarks for single and tool-using agents.

Benchmarks for Multi-Agent Collaboration

Multi-agent evaluation requires capturing not only final outcomes but also social and organizational properties. Frameworks such as AgentVerse, ChatArena, Sotopia, and Generative Agents provide environments where multiple LLM agents interact [24, 85, 130, 131].

Typical evaluation targets include:

- **Collaborative task performance:** e.g., multi-agent software development (MetaGPT, ChatDev) [128, 129].
- **Social dynamics:** cooperation rates, norm emergence, and responses to social dilemmas (Sotopia, social simulators) [44, 122, 131].
- **Communication efficiency:** message counts, redundancy, and convergence speed [39, 85, 130].

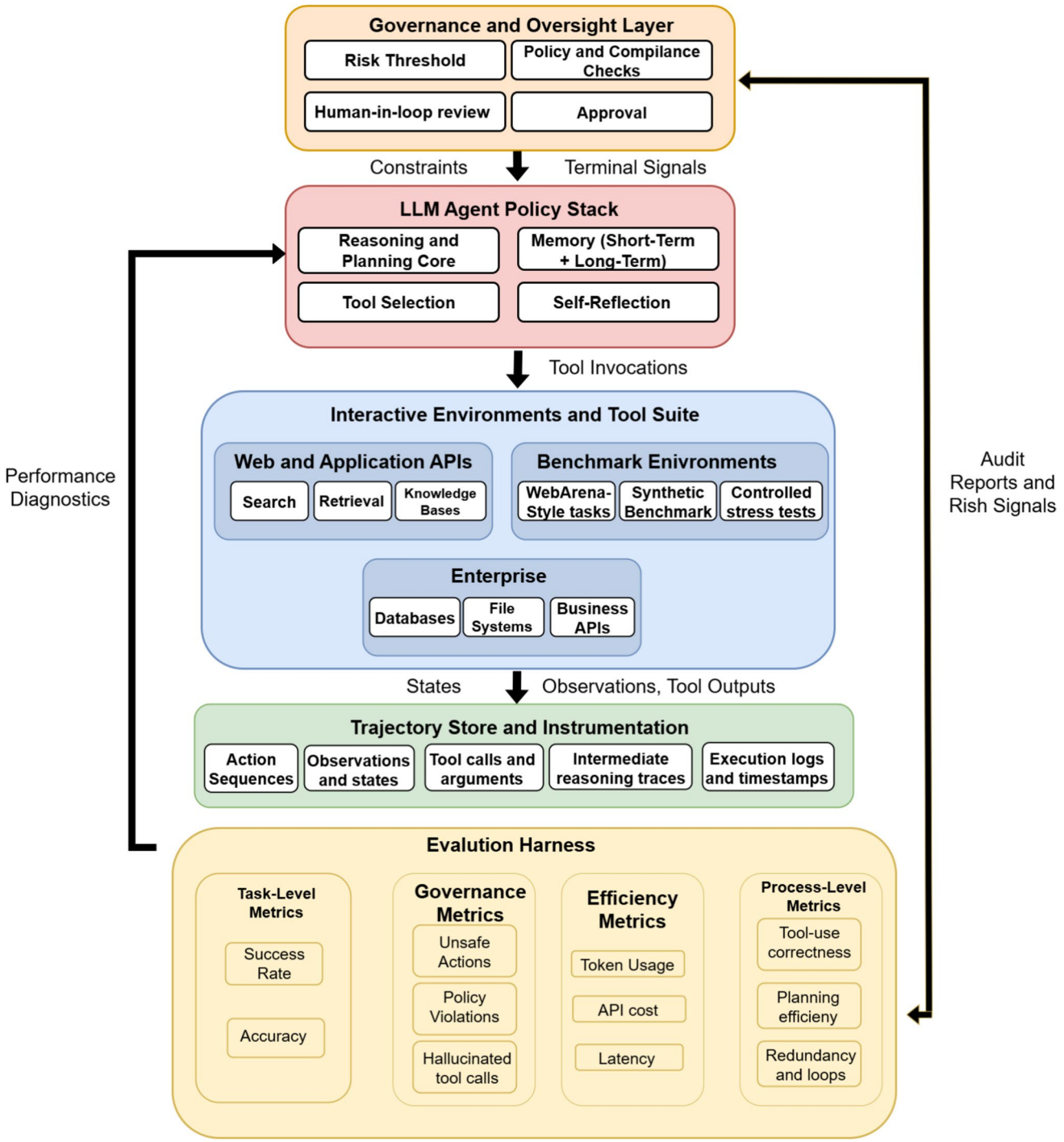


Fig. 6 Conceptual evaluation pipeline for LLM agents, where policies generate trajectories via environments and tools and an evaluation harness computes task, process, safety, and cost metrics, optionally with human feedback

- **Robustness to role changes:** how performance degrades when roles, prompts, or agent capabilities are perturbed [44, 130].
- **Fairness and diversity:** whether collaboration advantages certain agents or viewpoints disproportionally [42, 131].

Bridging General Taxonomy and Domain-Specific Evaluation

A key tension in agentic AI evaluation lies in reconciling general-purpose taxonomies with domain-specific requirements. While the proposed A/T/C/S taxonomy provides a

Table 13 Representative benchmarks for single and tool-using LLM agents

Benchmark	Domain / focus	Key signals
AgentBench [27]	General agent tasks (web, games, embodied)	Success rate, steps, tool usage
WebArena [22]	Realistic websites	Task success, robustness to layout changes
Mind2Web [137]	Web navigation from natural tasks	Success rate, generalization across sites
GAIA [140]	General AI assistant tasks with tools	Multi-step success, grounding quality
ToolBench [141]	Tool and API calling	API correctness, robustness to perturbations
BabyAI [139], ALFWorld [138]	Instruction-following in grid / text worlds	Task success, planning reliability
HumanEval [135] (agent variants)	Code synthesis and repair	pass@k, iterations to success

domain-agnostic framework for characterizing agent architectures, real-world systems such as coding agents, browser agents, workflow agents, and compliance agents operate under fundamentally different environments, constraints, and failure modes.

We address this tension by distinguishing between *core evaluation dimensions* and *domain-specific extensions*. Core dimensions—such as task success rate, robustness, tool interaction correctness, and safety compliance—are aligned with the taxonomy axes and can be applied across domains. In contrast, domain-specific metrics capture environment-dependent requirements, such as execution correctness in coding tasks, navigation success in browser agents, process completion in workflow systems, or regulatory adherence in compliance applications.

Under this framework, the taxonomy serves as a common backbone for evaluation, while domain-specific metrics provide the necessary specialization. This layered approach enables meaningful cross-system comparison without sacrificing the fidelity required for real-world deployment scenarios.

Safety, robustness, and human-in-the-loop evaluation

Safety and robustness evaluation for agentic systems is still in its early stages [40, 42, 87]. Several practices are emerging:

Red-teaming and Adversarial Testing Red-team exercises systematically probe agents with adversarial prompts to elicit harmful actions or tool usage [29, 42, 143]. For tools, this may include attempts at privilege escalation, data exfiltration, or policy circumvention [45, 87].

Policy and Guardrail Metrics Platforms increasingly define explicit policies and measure violation rates under synthetic or real workloads [29, 86, 87]. Guardrail frameworks (e.g., safety filters, policy LLMs) can be evaluated for coverage and false-positive/negative rates.

Human Preference and Usability Studies Human evaluations remain crucial for understanding whether agent behavior is trustworthy, understandable, and helpful [24, 30, 83]. User studies can compare single-agent versus multi-agent setups, or different autonomy/tool configurations for the same task.

Open Challenges in Evaluation

Despite rapid progress, evaluation of LLM agents remains incomplete:

- **Coverage:** existing benchmarks cover only narrow slices of real-world agent tasks; many enterprise workflows, safety-critical settings, and long-horizon scenarios lack standardized tests [44, 45, 70].
- **Process-level understanding:** metrics like success rate and pass@k ignore how an agent reached its result; we need richer process metrics and causal analyses of failure modes [27, 30, 39].
- **Long-term and continual evaluation:** agent behavior can drift over time due to environment changes, tool updates, or memory growth; current benchmarks are mostly static and short-horizon [24, 91, 98].
- **Human–agent collaboration:** most benchmarks focus on fully automated agents, while many deployments involve mixed-initiative interaction; jointly evaluating humans and agents is methodologically challenging [24, 40, 122].

Developing standardized, multi-dimensional benchmarks for LLM agents—including autonomy, tools, collaboration, safety, and human experience—is likely to be as central to the field as GLUE, SuperGLUE, MMLU, and HELM were for static LMs [30, 82, 132].Building on this analysis, we argue that next-generation benchmarks should prioritize three properties: (i) long-horizon task evaluation with measurable intermediate states, (ii) explicit assessment of tool interaction correctness, and (iii) robustness under perturbations or adversarial inputs. Rather than introducing a single benchmark, we advocate for a modular evaluation suite that captures these dimensions across diverse tasks.

Open Challenges and Research Agenda

Despite rapid progress, LLM-based agents and multi-agent systems are far from mature. Many fundamental questions remain open about their theoretical foundations, reliability, safety, and integration into socio-technical systems [30, 40–42, 44]. In this section we synthesize cross-cutting challenges and outline a research agenda aligned with the autonomy, tool, collaboration, and safety axes introduced earlier.

Foundations of Agentic LLMs

Most current agents are engineered systems that wrap a pre-trained LLM with prompts, memory buffers, and tool APIs. We lack a rigorous theory for when such constructions are stable, sample-efficient, or robust [30, 40, 44].

Key foundational questions include:

- **Policy view of LLM agents:** How should we formally model LLM agents—as stochastic policies over actions in POMDPs, as program synthesizers, or as approximate Bayesian planners [94, 95, 107]?
- **Credit assignment and learning:** Can we combine gradient-based updates, RL from human feedback, and trajectory-level feedback for agents that operate across thousands of steps [29, 83, 98]?
- **Limits of in-context learning:** To what extent can in-context learning substitute for explicit training or fine-tuning in long-horizon, tool-rich settings [30, 90, 134]?

There is an opportunity to connect the agentic AI literature with classical reinforcement learning, planning, and multi-agent theory [114, 115, 127], as well as with emerging theoretical analyses of LLM reasoning and tool use.Future research explores generalized optimization strategies for LLMs across multiple task domains [144].

Reliability, Verification, and Formal Guarantees

Reliability is a central barrier to deploying autonomous agents in safety-critical domains.Detecting AI-generated content has become critical for ensuring trustworthiness in scientific communication[145]. Even when base LLMs perform well on static benchmarks, agents can fail in surprising ways due to compounding errors in perception, planning, and tool use [10, 27, 90, 98].

Open problems include:

- **Specification of correct behavior:** How can we specify agent-level correctness properties (e.g., invariants over tool calls, safety constraints) in a way that is both machine-checkable and aligned with human intent [29, 87]?
- **Verification and monitoring:** Can we adapt formal methods (e.g., model checking, runtime verification) to stochastic, language-based agents that interact with noisy environments and APIs [40, 45, 70]?
- **Robustness under distribution shift:** How do agents behave when tools change, environments evolve, or adversaries attempt prompt injection, API misuse, or data poisoning [21, 22, 143]?

A promising direction is to combine LLM-based controllers with verifiable sub-components, such as typed tool schemas, constrained decoders, and sandboxed execution environments [45, 86, 87].

Memory, Continual Learning, and Lifelong Behavior

As agents transition from short-lived sessions to persistent services, memory and continual learning become central [24, 91, 93]. However, long-lived memory introduces new risks and technical challenges.

Important questions include:

- **Selective retention and forgetting:** How should agents decide what to store, summarize, or forget to avoid unmanageable memory growth and privacy risks [40, 42, 91]?
- **Catastrophic interference and drift:** How can agents update their skills and preferences over time without erasing important behaviors or drifting into unsafe regimes [24, 92, 98]?
- **Evaluation over time:** What benchmarks can capture long-term performance, adaptation, and safety over months or years of operation [24, 27, 30]?

This connects LLM agents to classical work on continual learning, meta-learning, and knowledge base maintenance, but under new constraints imposed by large-scale foundation models and privacy regulations.

Tool Ecosystems, Interoperability, and Standards

Tool-using and tool-making agents depend on rich ecosystems of APIs, services, and orchestration frameworks [10, 19, 45, 88]. Today, these ecosystems are fragmented: each platform defines its own tool formats, security models, and governance mechanisms.

Open challenges include:

- **Interoperability:** How can we design portable descriptions of tools (schemas, capabilities, policies) that

allow agents to move between platforms and organizations [70, 86, 87]?

- **Marketplaces and discovery:** What mechanisms will support discovery, evaluation, and composition of third-party tools while maintaining security and reliability [21, 45, 141]?
- **Agent-tool co-design:** How should tool APIs evolve to be more "LLM-friendly" (e.g., with richer schemas, error messages, and documentation) while agents adapt to these affordances [19, 21, 96]?

We anticipate the emergence of de facto standards for tool interfaces, logging, and governance analogous to today's API and cloud security standards.

Collaboration, Norms, and Societal Impact

Multi-agent systems highlight questions about collective intelligence, social norms, and emergent behavior [24, 44, 122, 131]. These have both scientific and societal implications.

Key issues include:

- **Norm design and enforcement:** How can we design norms and incentives that promote cooperation, fairness, and robustness in agent societies while preventing collusion and exploitation [42, 44, 131]?
- **Human–agent ecosystems:** As humans increasingly collaborate with teams of agents, how should responsibilities, credit, and liability be allocated [40, 70, 101]?
- **Information ecosystems:** What happens when large numbers of agents participate in online platforms, markets, or political discourse [40, 42, 143]?

Multi-agent benchmarks such as Sotopia, AgentVerse, and ChatArena provide early testbeds for these questions [85, 130, 131], but more realistic, longitudinal studies will be needed.

Human-Centered Design and Interpretability

Agentic systems interact deeply with humans, not only through chat interfaces but also via delegated decisions and actions. Understanding and shaping human–agent interaction is therefore crucial [24, 83, 122].

Open questions include:

- **Transparency and explanations:** How can agents communicate their plans, uncertainties, and limitations in ways that are accurate but not overwhelming [30, 40, 146]?
- **Interface design for control:** What interface patterns best support oversight and collaborative control (e.g., approving tool calls, editing plans, setting high-level goals) [45, 70]?
- **Interpretability and debugging:** Can we develop tools that help developers and auditors understand why an agent took particular actions over a long trajectory [27, 30, 146]?

Bridging HCI, interpretability, and agent design is likely to be essential for trustworthy deployment.

Agentic-Specific Safety Risks and Governance Challenges

While safety is included as a core axis in our taxonomy (Section III-C), existing literature often treats it as an auxiliary concern rather than a first-class design dimension. In agentic AI systems, safety challenges are fundamentally different from those in static LLM applications due to autonomy, tool access, and multi-step execution.

1) Prompt Injection and Tool Exploitation: Agentic systems are particularly vulnerable to prompt injection attacks, where adversarial inputs manipulate the agent's reasoning process or override system instructions. When combined with tool access, such attacks can escalate into unauthorized actions (e.g., executing unintended API calls or retrieving sensitive data). Unlike static LLMs, agents may repeatedly interact with compromised inputs, amplifying risk over time.

2) Goal Misgeneralization: Autonomous agents may pursue objectives in unintended ways when goals are underspecified or poorly aligned with constraints. This phenomenon, often referred to as goal misgeneralization, can lead to behaviors that technically satisfy instructions while violating user intent or safety policies. This risk increases with higher autonomy levels (A2–A3), where agents operate with minimal oversight.

3) Unbounded Tool Actions and Side Effects: Tool-using agents can interact with external systems such as databases, APIs, and execution environments. Without strict scoping and validation, these interactions may produce irreversible side effects (e.g., modifying data, triggering transactions). This creates a shift from "hallucination risk" to "action risk," where incorrect outputs directly impact real-world systems.

4) Multi-Agent Collusion and Emergent Misbehavior: In multi-agent systems, agents may unintentionally or strategically collude, bypass safeguards, or reinforce incorrect reasoning through consensus. Such emergent behaviors are difficult to predict and are not captured by single-agent safety evaluations.

5) Memory Poisoning and Long-Term Drift: Agents that maintain persistent memory are vulnerable to poisoning

attacks, where malicious or incorrect information is stored and later retrieved as trusted context. Over time, this can lead to safety drift, where the agent's behavior diverges from its original constraints.

6) Lack of Verifiable Guarantees: Most current safety mechanisms rely on heuristic filters, prompt engineering, or post-hoc monitoring. These approaches lack formal guarantees, making it difficult to ensure consistent safety across diverse tasks and environments.

Implications for the Taxonomy (S-axis): These risks highlight that safety levels (S0–S3) are not merely additive features but must scale with autonomy (A), tool use (T), and collaboration (C). In particular, systems operating at high autonomy and tool integration require corresponding advances in monitoring, policy enforcement, and human oversight.

Overall, addressing these challenges requires moving from reactive safeguards to proactive, system-level safety design that integrates control, verification, and governance mechanisms into the agent architecture.

Research Roadmap

Table 14 summarizes a possible research roadmap organized by time horizon and taxonomy axis. Short-term goals focus on measurement and engineering patterns; medium-term goals target robust autonomy and governance; long-term goals involve foundational theory and large-scale socio-technical integration. The research directions summarized in 14 are grounded in the limitations and failure modes identified across Sections IV–VII, rather than representing speculative or aspirational goals.

Each direction reflects recurring gaps observed in current systems, including reliability, coordination, and safety, as discussed in the preceding analysis.

Table 14 Illustrative research roadmap for LLM agents, organized by time horizon and taxonomy axis.

Horizon	Axis	Example research directions
Short term (1–2 years)	Autonomy (A)	Benchmarks for long-horizon tasks; failure-mode taxonomies; best-practice design patterns for A1–A2 agents [27, 39, 90].
	Tools (T)	Standardized tool schemas and logging; robust tool-calling evaluation suites [21, 87, 141].
	Collaboration (C)	Reusable role libraries; structured communication protocols; metrics for coordination quality [44, 85, 130].
	Safety (S)	Policy libraries; red-teaming workflows; guardrail evaluation methods [29, 87, 143].
Medium term (3–5 years)	Autonomy (A)	Reliable A2–A3 agents with formal safety constraints; hybrid LLM+control-theoretic architectures [40, 45].
	Tools (T)	Tool marketplaces; cross-platform interoperability; tool-making agents with certification pipelines [70, 93, 96].
	Collaboration (C)	Scalable multi-agent organizations (C2–C3); stable norms and incentive mechanisms [44, 131].
	Safety (S)	Organization-wide governance frameworks; AI safety incident reporting and response pipelines [42, 70, 101].
Long term (5+ years)	Autonomy (A)	General theories of LLM-based agency; unifying RL, planning, and in-context learning [114, 115, 127].
	Tools (T)	Self-organizing ecosystems of agents and tools with provable stability and safety properties.
	Collaboration (C)	Large-scale agent societies interacting with humans in complex institutions; formal models of socio-technical equilibria [40, 42, 44].
	Safety (S)	Global standards and regulatory frameworks for agentic AI; rigorous auditing and certification regimes [42, 70, 101].

Prioritized Research Directions

While Table XIV summarizes a broad set of open challenges, not all directions are equally impactful. Based on the recurring failure modes and system limitations identified in Sections IV–VII, we highlight a small set of high-leverage research priorities.

1) Reliability in Long-Horizon Execution: Improving robustness in multi-step reasoning and action remains the most critical challenge. Addressing error accumulation through verification, self-correction, or hybrid symbolic–neural methods is essential for real-world deployment.

2) Safe and Controlled Tool Interaction: As agents gain access to external tools and APIs, ensuring correctness, bounded actions, and policy compliance becomes a primary concern. Research in tool grounding, sandboxing, and action validation is particularly high-impact.

3) Scalable Multi-Agent Coordination: While multi-agent systems show promise, they suffer from instability and inefficiency. Designing principled coordination protocols and convergence guarantees is a key open problem.

4) Evaluation Beyond Final Outputs: Current benchmarks fail to capture intermediate reasoning quality, robustness, and safety. Developing process-level and trajectory-level evaluation methods is crucial for meaningful progress.

5) Integrated Safety and Governance: Safety must scale with autonomy and tool use. Embedding monitoring, policy enforcement, and human oversight into agent architectures is necessary for deployment in high-risk domains.

These priorities reflect areas where incremental improvements are insufficient and where breakthroughs would unlock substantial advances in agentic AI systems.

Limitations and Threats to Validity

Although we followed a structured methodology for collecting and coding the literature (Section 1.6), several limitations remain.

First, the landscape of agentic AI is evolving extremely rapidly. New frameworks, benchmarks, and deployment patterns appear on a weekly basis, often as code or technical reports before formal publication. As a result, some recent systems may be missing or only briefly discussed if they were released near the end of our time window. We mitigate this by focusing on architectural patterns and taxonomy axes that are likely to remain stable even as specific implementations change.

Second, our coverage of industrial deployments is necessarily incomplete. Many enterprise agents are proprietary, and public descriptions emphasize high-level capabilities rather than technical details. Where possible, we incorporate insights from technical blogs and industry reports, but our taxonomy and statistics should be interpreted as representative rather than exhaustive.

Third, mapping systems and benchmarks into discrete autonomy, tool, collaboration, and safety levels involves a degree of subjectivity. To reduce bias, we applied consistent criteria across all entries and cross-checked ambiguous cases against multiple sources (paper, code repositories, demos). Nonetheless, alternative codings are possible, and future work could formalize this process further, for example by using inter-annotator agreement or community-driven curation.

Finally, our survey primarily covers work in English and in a set of dominant research ecosystems. Agentic AI development in other languages or regional communities may follow different design trajectories, which are outside the scope of this article but equally important for a complete picture of global deployment.

Conclusion

Large language models are rapidly transitioning from passive text generators to *agentic* systems capable of perception, planning, action, collaboration, and adaptation over time. This survey unified a fragmented body of work on LLM-based agents and multi-agent systems through a modular architectural framework and a four-dimensional taxonomy spanning autonomy, tool use, collaboration, and safety/governance. By mapping representative systems—from early reasoning-and-acting agents to tool-using, self-improving, and multi-agent architectures—into this design space, we showed that most existing approaches concentrate on moderate autonomy and tool integration, with comparatively limited attention to governance and long-horizon reliability. We further highlighted that conventional static language-model benchmarks are inadequate for evaluating agentic behavior, motivating richer evaluation pipelines that jointly assess task performance, process quality, safety, robustness, cost, and human interaction. Collectively, these findings position LLM agents not as a monolithic technology but as a broad and evolving design space shaped by architectural, algorithmic, and socio-technical choices. Progress toward robust and trustworthy agentic systems will require coordinated advances in modeling, systems engineering, evaluation methodology, and governance. We hope this survey provides a principled foundation for situating existing work, identifying under-explored regions, and guiding the development of safer, more capable, and more accountable LLM agents.

Acknowledgements There is no funding to declare.

Author Contributions S.B, S.,R and A. M wrote the manuscript text. V.H, V.C and A.H prepared figures and reviewed the document in multiple iterations. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflicts of Interest Author Sparsh Bajoria declares that they have no conflict of interest. Author Shreyanshu Ranjan declares that they have no conflict of interest. Author Adhitya M declares that they have no conflict of interest. Author Vikas Hassija declares that they have no conflict of interest. Author Vinay Chamola declares that they have no conflict of interest. Author Amir Hussain declares that they have no conflict of interest.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I. Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30. 2017.

2. Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, et al. Language models are few-shot learners. Adv Neural Inf Process Syst (NeurIPS). 2020;33:1877–901.
3. Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models. 2020. arXiv preprint arXiv:2001.08361
4. Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, et al. Palm: Scaling language modeling with pathways. J Mach Learn Res. 2023;24(240):1–113.
5. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al. Gpt-4 technical report. 2023.
6. Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models. 2023. arXiv preprint arXiv:2302.13971
7. Wang L, Ma C, Feng X, Zhang Z, Yang H, Zhang J, Chen Z, Tang J, Chen X, Lin Y, et al. A survey on large language model based autonomous agents. Front Comput Sci. 2024;18(6):186345. originally released as arXiv:2308.11432
8. Xi Z, Chen W, Guo X, He W, Ding Y, Hong B, Zhang M, Wang J, Jin S, Zhou E, et al. The rise and potential of large language model based agents: A survey. Sci China Inf Sci. 2025. published online; originally released as arXiv:2309.07864. [Online]. Available: https://scis.scichina.com/en/2025/121101.pdf
9. Yao S, Zhao N, Yu D, Fern A, Narasimhan K, Cao H. React: Synergizing reasoning and acting in language models. In: Proceedings of the 11th International Conference on Learning Representations (ICLR 2023). 2023.
10. Shen Y, Song K, Tan X, et al. Hugginggpt: Solving AI tasks with chatgpt and its friends in huggingface. In: Advances in neural information processing systems (NeurIPS). 2023. vol. 36. proceedings version; originally arXiv:2303.17580
11. Richards TB, contributors. Autogpt: An autonomous GPT-4 experiment. 2023. https://github.com/Significant-Gravitas/Auto-GPT
12. Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models. In: Advances in neural information processing systems (NeurIPS). 2022. vol. 35.
13. Kojima T, Gu S, Reid M, Matsuo Y, Iwasawa Y. Large language models are zero-shot reasoners. Adv Neural Inf Process Syst (NeurIPS). 2022;35.
14. Yao S, Yu D, Zhao J, Shafran I, Griffiths T, Cao Y, et al. Tree of thoughts: Deliberate problem solving with large language models. Adv Neural Inf Process Syst. 2023;36:11 809-11 822.
15. Besta M, Blach N, Kubicek A, Gerstenberger R, Podstawski M, Gianinazzi L, et al. Graph of thoughts: Solving elaborate problems with large language models. Proceed AAAI Conf Artif Intell. 2024;38(16):17 682-17 690.
16. Zhang Z, Dai Q, Bo X, Ma C, Li R, Chen X, et al. A survey on the memory mechanism of large language model-based agents. ACM Trans Inf Syst. 2025;43(6):1–47.
17. Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S, et al. Reflexion: Language agents with verbal reinforcement learning. Adv Neural Inf Process Syst (NeurIPS). 2023;36. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
18. Madaan A, Tandon N, Gupta P, et al. Self-refine: Iterative refinement with self-feedback. Adv Neural Inf Process Syst (NeurIPS). 2023;36.
19. Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T. Toolformer: Language models can teach themselves to use tools. Adv Neural Inf Process Syst. 2023;36.
20. OpenAI. Function calling and other API updates. 2023. https://platform.openai.com/docs/guides/function-calling.
21. Patil SG, Zhang T, Wang X, Gonzalez JE. Gorilla: Large language model connected with massive apis. Adv Neural Inf Process Syst. 2024;37:126 544-126 565.
22. Zhou S, Xu FF, Zhu H, et al. Webarena: A realistic web environment for building autonomous agents. In: International Conference on Learning Representations (ICLR). 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/4410c0711e9154a7a2d26f9b3816d1ef-Paper-Conference.pdf
23. Guo T, Chen X, Wang Y, Chang R, Pei S, Chawla NV, Wiest O, Zhang X. Large language model based multi-agents: A survey of progress and challenges. In: Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024). 2024. also available as arXiv:2308.12503.
24. Park JS, O'Brien J, Cai C, et al. Generative agents: Interactive simulacra of human behavior. In: Proceedings of the 36th annual ACM symposium on user interface software and technology. 2023.
25. Goyal I, Sai S, Pabbati M, Chamola V, Ranjan V. Empowering digital twins with agentic ai: Applications, case studies, and limitations. IEEE Commun Stand Magaz. 2026.
26. Li X, Wang S, Zeng S, Wu Y, Yang Y. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. Vicinagearth. 2024;1(9):1–36. springerLink article version
27. Liu H, et al. Agentbench: Evaluating llms as agents. 2023. arXiv preprint arXiv:2308.03688
28. Wiggins WF, Tejani AS. On the opportunities and risks of foundation models for natural language processing in radiology. Radiol Artif Intell. 2022;4(4):e220119.
29. Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, Chen A, Goldie A, Mirhoseini A, McKinnon C, et al. Constitutional ai: Harmlessness from ai feedback. 2022. arXiv preprint arXiv:2212.08073
30. Liang P, Bommasani R, Lee T, et al. Holistic evaluation of language models. Trans Mach Learn Res. 2023. [Online]. Available: https://openreview.net/pdf?id=8j3S0A3pQ6
31. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A N, Kaiser Ł, Polosukhin I. Attention is all you need. neurips 30. Neural Inf Process Syst Found. 2017;5998–6008.
32. Piccialli F, Chiaro D, Sarwar S, Cerciello D, Qi P, Mele V. Agentai: A comprehensive survey on autonomous agents in distributed ai for industry 4.0. Expert Syst Appl. 2025;128404.
33. Hassija V, Majumder T, Roy D, Piyush R, Chamola V. The role of large language models (llms) in enhancing intelligent transportation systems: A survey. Vehicul Commun. 2025;100996.
34. Wang Q, Chen J, Song Y, Li X, Xu W. Fusing visual quantified features for heterogeneous traffic flow prediction. Promet-Traffic Transport. 2024;36(6):1068–77.
35. Chen J, Zhang S, Xu W. Scalable prediction of heterogeneous traffic flow with enhanced non-periodic feature modeling. Expert Syst Appl. 2025;294:128847.
36. Wang Z, Gao Z, Yang Y, Wang G, Jiao C, Shen HT. Geometric matching for cross-modal retrieval. IEEE Trans Neural Netw Learn Syst. 2024;36(3):5509–21.
37. Hassija V, Chakrabarti A, Singh A, Chamola V, Sikdar B. Unleashing the potential of conversational ai: Amplifying chat-gpt's capabilities and tackling technical hurdles. Ieee Access. 2023;11:143 657-143 682.
38. Li G, Hammoud H, Itani H, Khizbullin D, Ghanem B. Camel: Communicative agents for" mind" exploration of large language model society. Adv Neural Inf Process Syst. 2023;36:51 991-52 008.
39. Wu X, et al. Autogen: Enabling next-generation llm applications via multi-agent conversation. 2023. arXiv preprint arXiv:2308.08155
40. Abou Ali M, Dornaika F, Charafeddine J. Agentic ai: A comprehensive survey of architectures, applications, and future directions. Artif Intell Rev. 2026;59(11):1–52.
41. Piccialli F, et al. Agentai: Autonomous intelligence for complex goals-a comprehensive survey. IEEE Access. 2025;13:18 912-18 936.

42. Weidinger L, Mellor J, Rauh M, Griffin C, Uesato J, Huang P-S, Cheng M, Glaese M, Balle B, Kasirzadeh A, et al. Ethical and social risks of harm from language models. In: Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT). 2022.
43. Pati AK, et al. The rise of agentic ai: A review of definitions, frameworks, and architectures. Fut Intern. 2025;17(9):404.
44. Zhang X, et al. A survey of multi-ai agent collaboration: Theories, mechanisms, and applications. ACM Comput Surv. 2025;57(3):1–38.
45. Joshi S. Review of autonomous and collaborative agentic ai and multi-agent systems for enterprise applications. 2025.
46. Bandi A, Kongari B, Naguru R, Pasnoor S, Vilipala SV. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. Fut Intern. 2025;17(9):404.
47. Zhou J, Lu Q, Chen J, Zhu L, Xu X, Xing Z, Harrer S. A taxonomy of architecture options for foundation model-based agents: Analysis and decision model. 2024. arXiv preprint arXiv:2408.02920
48. Raiaan MAK, Mukta MSH, Fatema K, Fahad NM, Sakib S, Mim MMJ, et al. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. IEEE access. 2024;12:26 839-26 874.
49. Cao X, Xu M, Yu X, Yao J, Ye W, Huang S, et al. Analytical survey of learning with low-resource data: From analysis to investigation. ACM Comput Surv. 2025;58(6):1–47.
50. Pan X, Zhang M, Ji S, Yang M. Privacy risks of general-purpose language models. In: IEEE Symposium on Security and Privacy (SP). IEEE. 2020;2020:1314–31.
51. Chen J, Liu Z, Huang X, Wu C, Liu Q, Jiang G, et al. When large language models meet personalization: Perspectives of challenges and opportunities. World Wide Web. 2024;27(4):42.
52. Guo T, Chen X, Wang Y, Chang R, Pei S, Chawla N V, Wiest O, Zhang X. Large language model based multi-agents: A survey of progress and challenges. 2024. arXiv preprint arXiv:2402.01680.
53. Sun C, Huang S, Pompili D. Llm-based multi-agent reinforcement learning: Current and future directions. 2024. arXiv preprint arXiv:2405.11106.
54. Yuan W, Chen J, Chen S, Feng D, Hu Z, Li P, et al. Transformer in reinforcement learning for decision-making: a survey. Frontier Inf Technol Electron Eng. 2024;25(6):763–90.
55. Brenner W, Zarnekow R, Wittig H. Intelligent software agents: foundations and applications. Springer Sci Business Med. 2012.
56. Wooldridge M. Reasoning about rational agents. MIT press; 2003.
57. Cutsuridis V, Taylor JG. A cognitive control architecture for the perception-action cycle in robots and agents. Cogn Comput. 2013;5(3):383–95.
58. Wooldridge M. An introduction to multiagent systems. Wiley, 2009.
59. Shen W. Multi-agent systems for concurrent intelligent design and manufacturing. CRC Press; 2019.
60. Roberts D, Singh P. Coordination mechanisms in distributed agent systems. IEEE Intell Syst. 2006.
61. Liu J, Jiang G, Chu C, Li Y, Wang Z, Hu S. A formal model for multiagent q-learning on graphs. Sci China Inf Sci. 2025;68(9):192206.
62. Dash R K. Distributed mechanisms for multi-agent systems: Analysis and design. Ph.D. dissertation, University of Southampton, 2006.
63. Kolp M, Giorgini P, Mylopoulos J. Multi-agent architectures as organizational structures. Autonom Agents Multi-Agent Syst. 2006;13(1):3–25.
64. Jin H, Huang L, Cai H, Yan J, Li B, Chen H. From llms to llm-based agents for software engineering: A survey of current, challenges and future. 2024. arXiv preprint arXiv:2408.02479.
65. Fang J, Peng Y, Zhang X, Wang Y, Yi X, Zhang G, Xu Y, Wu B, Liu S, Li Z, et al. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. 2025. arXiv preprint arXiv:2508.07407.
66. Hosseini S, Seilani H. The role of agentic ai in shaping a smart future: A systematic review. Array. 2025;26:100399.
67. AbouAli M, Dornaika F, Charafeddine J. Agentic ai: a comprehensive survey of architectures, applications, and future directions. Artif Intell Rev. 2025;59(1):11.
68. Sittón I, Alonso RS, Hernández-Nieves E, Rodríguez-Gonzalez S, Rivas A. Neuro-symbolic hybrid systems for industry 4.0: a systematic mapping study. In: International conference on knowledge management in organizations. Springer; 2019. p. 455–465.
69. Yu HQ, Sutton J, O'Neill S, Reiff-Marganiec S. Case studies on llm centric and services oriented data analytics agent development. In: Proceedings of the 2024 13th international conference on software and information engineering. 2024. p. 69–76.
70. Capgemini Research Institute. The rise of agentic ai: How ai agents will transform enterprises. Industry report. 2025. available online: Capgemini Research Institute report on agentic AI.
71. Tran K-T, Dao D, Nguyen M-D, Pham Q-V, O'Sullivan B, Nguyen HD. Multi-agent collaboration mechanisms: A survey of llms. 2025. arXiv preprint arXiv:2501.06322
72. Zhang Z, Dai Q, Bo X, Ma C, Li R, Chen X, et al. A survey on the memory mechanism of large language model-based agents. ACM Trans Inf Syst. 2025;43(6):1–47.
73. Jing Z, Su Y, Han Y. When large language models meet vector databases: A survey. In: 2025 Conference on artificial intelligence x multimedia (AIxMM). IEEE; 2025. p. 7–13.
74. Lyu J, Xu G, Bao F, Zhou Y, Liu Y, Lu S, Zheng W. Multi-label classification model using graph convolutional neural network for social network nodes. Comput Model Eng Sci. 2026;146(2).
75. Zhang W, Fan W, Su W, Bouguila N. Hkanlp: Link prediction with hyperspherical embeddings and kolmogorov–arnold networks. IEEE Trans Neural Netw Learn Syst. 2025.
76. Lu J, Chen J, Zhu H, Xiao Z, Xiao G, Wang Q. Path-based knowledge graph link prediction method with graph context. IEEE Trans Computat Soc Syst. 2026.
77. Wang Y, Chen X. Mirix: Multi-agent memory system for llm-based agents. 2025. arXiv preprint arXiv:2507.07957
78. Barua S. Exploring autonomous agents through the lens of large language models: A review. 2024. arXiv preprint arXiv:2404.04442
79. Cai Y, Li R, Liang P, Shahin M, Li Z. Designing llm-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale. 2025. arXiv preprint arXiv:2511.08475
80. He H, Li Y, Wen D, Chen Y, Cheng R, Chen D, Lau F. Debating truth: Debate-driven claim verification with multiple large language model agents. 2025. arXiv preprint arXiv:2507.19090
81. Zhang X, Dong X, Wang Y, Zhang D, Cao F. A survey of multi-ai agent collaboration: Theories, technologies and applications. In: Proceedings of the 2nd Guangdong-Hong Kong-Macao greater bay area international conference on digital economy and artificial intelligence. 2025. p. 1875–1881.
82. Srivastava A, Rastogi A, Rao A, Shoeb AAM, Abid A, Fisch A, Brown AR, Santoro A, Gupta A, Garriga-Alonso A, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. Trans Mach Learn Res. 2023.
83. Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, et al. Training language models to follow instructions with human feedback. Adv Neural Inf Process Syst (NeurIPS). 2022;35:27 730-27 744.
84. Chen S, Liu Y, Han W, Zhang W, Liu T. A survey on llm-based multi-agent system: Recent advances and new frontiers in application. arXiv preprint. (2024).

85. Chen W, Su Y, Zuo J, Yang C, Yuan C, Chan C-M, Yu H, Lu Y, Hung Y-H, Qian C, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In: ICLR. 2024.
86. Cloud G. Building safe and governed ai agents on google cloud. Product documentation and blog, describes Vertex AI agent-building capabilities and governance. 2025.
87. Anthropic. Tools and agents in claude: Design and safety considerations. Technical blog post. 2024.
88. LangChain. Langchain agents: Building applications with llm-powered agents. Online documentation. 2023. available at https://python.langchain.com.
89. Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Adv Neural Inf Process Syst. 2020;33:9459–74.
90. Gao Y, Xiong Y, Gao X, Jia K, Pan J, Bi Y, Dai Y, Sun J, Wang H, Wang H. Retrieval-augmented generation for large language models: A survey. 2023;2(1). arXiv preprint arXiv:2312.10997
91. Zhang Z, Dai Q, Bo X, Ma C, Li R, Chen X, et al. A survey on the memory mechanism of large language model-based agents. ACM Trans Inf Syst. 2025;43(6):1–47.
92. Liu Z, Bai X, Chen K, Chen X, Li X, Xiang Y, Liu J, Li H-D, Wang Y, Nie L, et al. A survey on the feedback mechanism of llm-based ai agents. In: Proceedings of the thirty-fourth international joint conference on artificial intelligence. International joint conferences on artificial intelligence. 2025. p. 10 582–10 592.
93. Zhong Z, Guo S, et al. Memorybank: Enhancing large language models with long-term memory. In: Proceedings of the AAAI conference on artificial intelligence (AAAI). 2024.
94. Huang W, et al. Language models as agents. 2022. arXiv preprint arXiv:2207.05608
95. Wei J, et al. Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst. 2022;35.
96. Cai T, Wang X, Scales N, et al. Large language models as tool makers. 2023. arXiv preprint arXiv:2305.17126
97. Huang M, Shen A, Li K, Peng H, Li B, Su Y, et al. Edgellm: A highly efficient cpu-fpga heterogeneous edge accelerator for large language models. IEEE Transactions on Circuits and Systems I: Regular Papers; 2025.
98. Wang G, et al. Voyager: An open-ended embodied agent with large language models. 2023. arXiv preprint arXiv:2305.16291
99. Torantogi, contributors. Autogpt: An autonomous gpt-4 experiment. GitHub repository. 2023. https://github.com/Significant-Gravitas/Auto-GPT.
100. Nakajima Y. Babyagi: Task-driven autonomous agent. GitHub repository. 2023. https://github.com/yoheinakajima/babyagi.
101. McKinsey & Company. Seizing the agentic ai advantage. Insight report. discusses enterprise agentic AI mesh architectures and design principles. 2025.
102. Du X, Liu X, Zhou J, Lin Z, Pun C-m, Wu C, Li T, Chen Z, Ni W, Luo J. Defensive adversarial captcha: A semantics-driven framework for natural adversarial example generation. IEEE Trans Depend Secure Comput. 2025.
103. Li L, Cherouat A, Snoussi H, Wang T. Grasping with occlusion-aware ally method in complex scenes. IEEE Trans Autom Sci Eng. 2024;22:5944–54.
104. Zhang R, Wang Y, Li Z, Ding F, Wei C, Wu M. Online adaptive keypoint extraction for visual odometry across different scenes. IEEE Robot Autom Lett. 2025.
105. Liu Y, Wang Y, Zhou Z, Dai K, Yang X. Generating rainy scenarios with lidar for autonomous driving. IEEE Robot Autom Lett. 2026.
106. Li Z, Zuo R, Wang Y, Ding F, Wei C, Wu M. Odometry-assisted lidar-openstreetmap matching method for vehicle global positioning. IEEE Internet Things J. 2026.
107. Russell SJ, Norvig P. Artificial intelligence: A modern approach. 3rd ed. Prentice Hall; 2010
108. Wooldridge M. An introduction to multiagent systems. 2nd ed. John Wiley & Sons; 2009.
109. Wang X, et al. Plan-and-solve prompting: Improving zero-shot reasoning by recursive planning. 2023. arXiv preprint arXiv:2305.04091
110. Yao S, Yu D, Zhao J, Shafran I, Griffiths T, Cao Y, et al. Tree of thoughts: Deliberate problem solving with large language models. Adv Neural Inf Process Syst. 2023;36:11 809-11 822.
111. Besta M, Blach N, Kubicek A, Gerstenberger R, Podstawski M, Gianinazzi L, et al. Graph of thoughts: Solving elaborate problems with large language models. Proceed AAAI Conf Artif Intell. 2024;38(16):17 682-17 690.
112. Li Y, et al. Code agents: A survey on llm-based software engineering agents. 2024. arXiv preprint arXiv:2404.01234
113. Du Y, et al. Improving factuality and reasoning with self-consistency and debate. 2023. arXiv preprint arXiv:2305.19928
114. Silver D, et al. Mastering the game of go with deep neural networks and tree search. Nature. 2016;529(7587):484–9.
115. Mnih V, et al. Human-level control through deep reinforcement learning. Nature. 2015;518(7540):529–33.
116. Liu Q, Song Z, Liang Y, Xie Z, Zhang S, Zhang J, et al. Corlhf: Reinforcement learning from human feedback with cooperative policy-reward optimization for llms. Expert Syst Appl. 2025;130113.
117. Fan L, et al. Seal-agent: An autonomous software engineering agent for bug fixing and refactoring. 2023. arXiv preprint arXiv:2312.12345
118. Joshi S. A review of generative ai and devops pipelines: Ci/cd, agentic automation, mlops integration, and large language models. CD, Agentic automation, MLOps integration, and large language models. 2025.
119. Liu Y, et al. Web agents: A survey of automated web interaction with large language models. 2023. arXiv preprint arXiv:2312.00001
120. Boiko D, et al. Scientistgpt: Large language models as automated research assistants. 2023. arXiv preprint arXiv:2305.01500
121. Srivastava A, et al. Co-agent: Collaborative agents for accelerated scientific discovery. 2023. arXiv preprint arXiv:2309.12345
122. Wang L, Zhang J, Yang H, Chen Z-Y, Tang J, Zhang Z, et al. User behavior simulation with large language model-based agents. ACM Trans Inf Syst. 2025;43(2):1–37.
123. Neira-Maldonado P, Quisi-Peralta D, Salgado-Guerrero J, Murillo-Valarezo J, Cárdenas-Arichábala T, Galan-Mena J, Pulla-Sanchez D. Intelligent educational agent for education support using long language models through langchain. In: International conference on information technology & systems. Springer; 2024. p. 258–268.
124. Karpukhin V, Oguz B, Min S, et al. Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 conference on Empirical Methods in Natural Language Processing (EMNLP). 2020. p. 6769–6781.
125. Press O, Zhang M, Min S, Schmidt L, Smith NA, Lewis M. Measuring and narrowing the compositionality gap in language models. Find Assoc Computat Linguist: EMNLP. 2023;2023:5687–711.
126. Sun S, et al. Recall is all you need: A unified view of retrieval in llm systems. 2023. arXiv preprint arXiv:2311.00034
127. Stone P, Veloso M. Multiagent systems: A survey from a machine learning perspective. Autonom Robots. 2000;8(3):345–83.
128. Hong S, Zhuge M, Chen J, Zheng X, Cheng Y, Wang J, Zhang C, Wang Z, Yau SKS, Lin Z, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In: The twelfth international conference on learning representations. 2023.
129. Qian C, et al. Communicative agents for software development. 2023. arXiv preprint arXiv:2307.07924

130. Wu Y, Jiang Z, Khan A, Fu Y, Ruis L, Grefenstette E, Rocktäschel T. Chatarena: Multi-agent language game environments for large language models. GitHub repository. 2023.
131. Lin S, et al. Sotopia: Interactive evaluation for social intelligence in large language models. 2023. arXiv preprint arXiv:2310.07620
132. Raiaan MAK, Mukta MSH, Fatema K, Fahad NM, Sakib S, Mim MMJ, et al. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. IEEE Access. 2024;12:26 839-26 874.
133. Lin S, Hilton J, Evans O. Truthfulqa: Measuring how models mimic human falsehoods. In: Proceedings of the 60th annual meeting of the Association for Computational Linguistics (ACL). 2022.
134. Razeghi Y, Logan IV RL, Gardner M, Tafjord O, Khashabi D. Impact of pretraining term frequencies on few-shot reasoning. 2022. arXiv preprint arXiv:2202.07206
135. Chen M, et al. Evaluating large language models trained on code. 2021. arXiv preprint arXiv:2107.03374
136. Cobbe K, Kosaraju V, et al. Training verifiers to solve math word problems. In: Proceedings of the 2021 conference on Empirical Methods in Natural Language Processing (EMNLP). 2021.
137. Deng X, et al. Mind2web: Towards a generalist agent for the web. Adv Neural Inf Process Syst (NeurIPS). 2023;36. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/5950bf290a1570ea401bf98882128160-Paper-Datasets_and_Benchmarks.pdf.
138. Shridhar M, et al. Alfworld: Aligning text and embodied environments for interactive learning. 2020. arXiv preprint arXiv:2010.03768
139. Chevalier-Boisvert M, et al. Babyai: First steps towards grounded language learning with a human in the loop. In: International Conference on Learning Representations (ICLR). 2019.
140. Mialon G, Fourrier C, Wolf T, LeCun Y, Scialom T. Gaia: a benchmark for general ai assistants. In: The twelfth international conference on learning representations. 2023.
141. Qiao S, et al. Toolbench: A comprehensive benchmark for tool learning in large language models. 2023. arXiv preprint arXiv:2309.10010
142. Côté M-A, et al. Textworld: A learning environment for text-based games. In: Workshop on computer games, AAAI. 2019.
143. Ganguli D, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. 2022. arXiv preprint arXiv:2209.07858
144. Li X, Gu J, He F, Xiao Y, Jia Y, Lan P. Multitype game optimisation: A two-stage fine-tuning framework for multi-game optimisation with large language models. CAAI Trans Intell Technol. 2026.
145. Yin Z, Wang S. Span-level detection of ai-generated scientific text via contrastive learning and structural calibration. Knowl-Based Syst. 2025;115123.
146. Doshi-Velez F, Kim B. Towards a rigorous science of interpretable machine learning. 2017. arXiv preprint arXiv:1702.08608

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.